

THE ULTIMATE DEVELOPER'S GUIDE TO

CORS

Border Control for the Web

Cross-Origin Resource Sharing from first principles — the same-origin policy it relaxes, the conversation between browser and server, the mistakes that turn it into a security incident, and how to get it right.



Access-Control-Allow-Origin: <https://app.harborpay.example>

SAME-ORIGIN POLICY · PREFLIGHT · CREDENTIALS · SECURE CONFIGURATION

8 hands-on labs · a zero-dependency lab server · real incidents · secure patterns

AUTHOR

Smiroh Dev

Smiroh Publishers
FIRST EDITION

The Ultimate Developer's Guide to CORS: Border Control for the Web

Smiroh Dev

PUBLISHER	Smiroh Publishers
EDITION	First edition
PUBLISHED	September 2026
FORMAT	Electronic book (PDF), 61 pages
ISBN (EB00K)	To be assigned
SUBJECT	Computers / Security / Web Development

Copyright © 2026 Smiroh Dev. All rights reserved.

Published by Smiroh Publishers. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

TRADEMARKS

All trademarks and registered trademarks appearing in this book are the property of their respective owners. They are used editorially, with no intention of infringement. The publisher is not associated with any product, vendor, or organization mentioned in this book. Harbor Pay and every character in the story passages are fictional.

STANDARDS REFERENCED

CORS is defined today in the WHATWG Fetch Living Standard, and the origin concept in RFC 6454. Those documents are the normative sources; where this book and a specification disagree, the specification governs. A reading list appears at the end.

ABOUT THE CASE STUDIES

The real-world incidents in Chapter 15 are summarized from public disclosures, advisories and published research, and are told at the level of what went wrong and how it was fixed. Details are as reported at the time; consult the original sources before citing them.

LIMIT OF LIABILITY AND DISCLAIMER OF WARRANTY

The information in this book is provided on an “as is” basis, without warranty of any kind, express or implied. While every precaution has been taken in its preparation, neither the author nor the publisher shall have any liability to any person or entity with respect to any loss or damage caused, or alleged to have been caused, directly or indirectly by the information contained in it. Browser behavior changes: verify any practice described here against current browser documentation before relying on it.

ABOUT THE LAB SERVER

The lab server is a teaching tool. It serves plain HTTP on `localhost`, keeps all state in memory, and must never be exposed to a network or used to protect anything real.

What is in this guide

Part I — A Web Without Walls

01	The launch that went dark	A story, a red error, and the question this book answers	0
02	Why browsers built walls	Cookies, ambient authority, and the birth of the same-origin policy	0
03	What exactly is an origin?	Scheme, host, port — and why “site” is something else	0
04	What the wall blocks, and what it doesn't	Sending, embedding, reading: the one rule that matters	0
05	Life before CORS	JSONP, proxies, document.domain, and the lessons they taught	0

Part II — How CORS Works

06	Enter CORS	A permission slip written in HTTP headers	0
07	Simple requests	Why some requests go straight through	0
08	The preflight	Asking permission before acting	0
09	Credentials and cookies	The strictest rules in CORS, and why they exist	0
10	Reading the response	Exposed headers, opaque responses, and silent errors	24
11	Debugging CORS like a pro	A flowchart, the error messages, and the tools	25

Part III — When the Wall Has Holes

12	Why CORS is a security priority	One header between a hostile page and your users' data	28
13	Configuration mistakes that matter	Six patterns that turn a policy into a hole, and their fixes	30
14	Attack patterns, drawn	Seven attack shapes, where to break each chain, and how to prepare	32
15	Case studies from the real world	Public incidents and research, and what each one teaches	37
16	CORS is not CSRF protection	What CORS never promised, and what does protect you	39

Part IV — Advanced Territory

17	Caches and Vary: Origin	The one-line header behind “it works sometimes”	41
18	The local network	Localhost services, routers, and Local Network Access	42
19	Beyond CORS	CORP, COEP, COOP, Fetch Metadata and cross-origin isolation	44
20	Designing a secure CORS policy	Architecture patterns, reference configurations, a checklist	46
21	Epilogue: the review	How the story ends, and what to carry with you	49

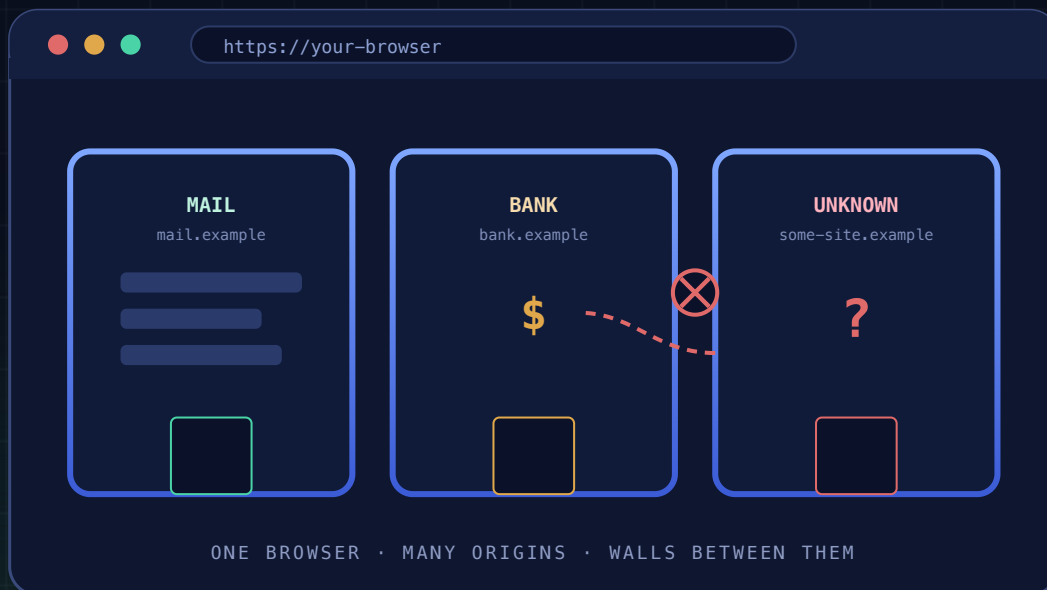
Part V — The Labs

—	Setup	51	7–8	Caching and allowlists	54
1–3	First contact	52	A	lab/server.js listing	55
4–6	Credentials and headers	53			
—	Glossary	Every term in one place			59
—	Further reading	The specifications, and the research behind the case studies			60

PART I

A Web Without Walls

Before a single CORS header makes sense, you need the thing it relaxes: the same-origin policy, the oldest and most important security boundary in your browser. This part tells the story of why it exists, what it guards, and the clever, dangerous hacks developers used before CORS arrived.



-
- 01 The launch that went dark · 02 Why browsers built walls · 03 What exactly is an origin?
04 What the wall blocks, and what it doesn't · 05 Life before CORS

The launch that went dark

Every developer meets CORS the same way: as a red line in the console, at the worst possible moment. This book starts there, too.

HARBOR PAY • LAUNCH NIGHT • 11:52 P.M.

You have been at Harbor Pay for six weeks. Tonight the new customer dashboard goes live: a fast single-page app at `app.harborpay.example`, talking to the company's API at `api.harborpay.example`. It worked perfectly on your laptop. It passed every test.

At 11:52 the deploy finishes. You open the dashboard. The balance card spins, and spins. You open DevTools and the console is a wall of red:

```
Access to fetch at 'https://api.harborpay.example/v1/balance' from origin  
'https://app.harborpay.example' has been blocked by CORS policy: No  
'Access-Control-Allow-Origin' header is present on the requested resource.
```

In the team channel, the suggestions arrive fast. “Just add `Access-Control-Allow-Origin: *`.” Then: “Star won't work with cookies. Make the server copy back whatever Origin it receives — that always works.” Someone pastes a one-line middleware snippet from a forum.

Your cursor hovers over the merge button. Then Ren, the security lead, joins the call. “Before anybody touches that header,” Ren says, “can someone tell me what it actually protects?”

Nobody answers. The launch waits.

The question this book answers

That silence is the reason this book exists. The error message tells you what is missing; it does not tell you what the missing thing *means*. The two “quick fixes” offered on the call make opposite trade-offs. The first shares public data with everyone, which is often fine. The second, copying back any Origin with cookies allowed, can hand every website on the internet the ability to read your signed-in customers' data. Both make the red line disappear. Neither is the right answer for a bank.

CORS in one sentence. Cross-Origin Resource Sharing is the protocol a server uses to tell the browser which other websites may read its responses — a controlled exception to a rule the browser otherwise enforces without mercy.

How this book is organized

We will follow the Harbor Pay team from that launch night to a security review three weeks later. Each chapter opens with a scene, then turns the scene into principles, diagrams, and a lab you can run on your own machine.

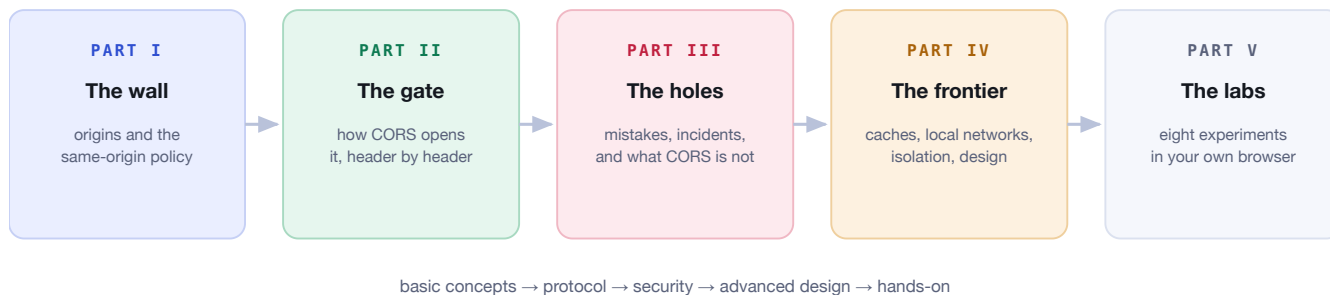


Figure 1.1 — The route through the book. Each part builds on the one before it; the labs can be run at any point after Chapter 6.

What you need

To read it

Comfort with HTTP requests and responses, and enough JavaScript to read a `fetch()` call. No security background is assumed.

To run the labs

Node.js 18 or later and any modern browser. The lab server has zero dependencies and prints its full source in Part V.

To use it at work

Chapter 20's checklist and reference configurations are written to be copied into a design review.

Three conventions

- **Story passages** appear on a warm background, like the one on the previous page. Harbor Pay and its people are fictional; the technology and the incidents are real.
- **Example domains** end in `.example`, a name reserved for documentation, so nothing in this book points at a real system.
- **Header names** are case-insensitive in HTTP. The book writes them in their usual capitalization, `Access-Control-Allow-Origin`, and lowercase in code, where Node.js presents them that way.

Where the story goes. Tonight's launch is fixed in Chapter 9. The harder question Ren asked — what the header protects — takes until Chapter 13 to answer fully, and it changes how the whole team designs APIs.

Why browsers built walls

Your browser is the most trusting program you run. It carries your identity to every site you are signed in to — automatically, on every request. That generosity is why the wall has to exist.

LAUNCH NIGHT • 12:10 A.M.

Ren shares a screen with two browser tabs. One is Harbor Pay's dashboard, signed in. The other is a recipe blog. *“Both of these pages run code in the same program, on the same machine, with the same cookies,”* Ren says. *“So why can't the recipe blog read your bank balance?”*

Ambient authority: the browser's great convenience

When you sign in to a website, it usually hands your browser a **session cookie**. From then on, the browser attaches that cookie to every request it sends to that site — no matter which page, tab, or script caused the request. Security engineers call this **ambient authority**: your credentials are simply in the air, applied automatically.

It is what makes the web feel seamless. It is also a loaded gun. If any page could send a request to your bank *and read the answer*, then every page you visit could read your statements, your email, your medical portal — using your own cookies.

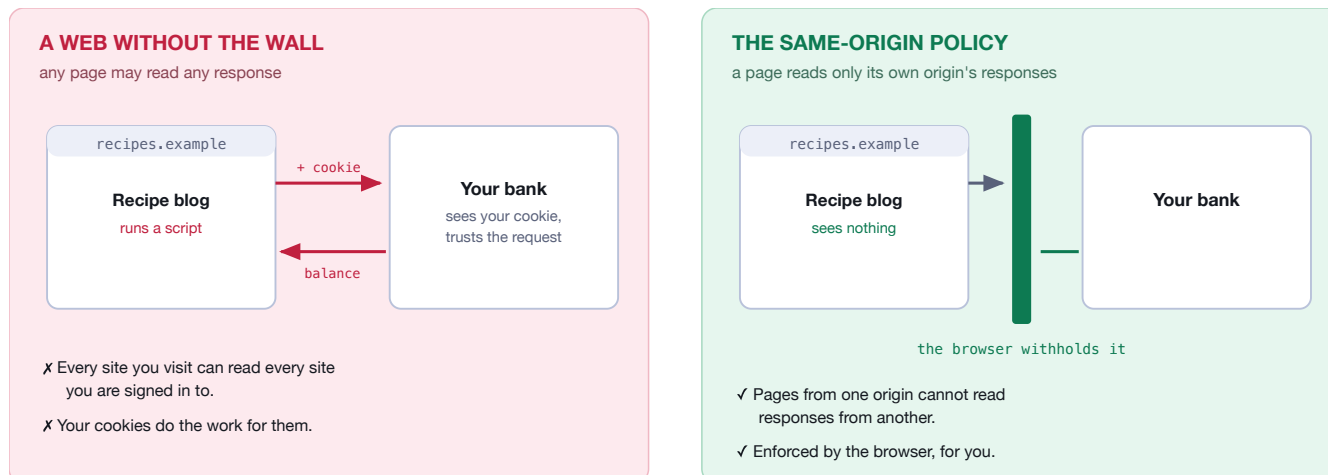


Figure 2.1 — Why the wall exists. Cookies make requests *authoritative*; the same-origin policy makes sure only the right page can read what those requests return.

The rule arrives

JavaScript shipped in Netscape Navigator 2 in the mid-1990s, and within months the browser makers faced the problem in Figure 2.1. Their answer was the **same-origin policy** (SOP): a script may freely read data that comes from its own origin, and is cut off from data that comes from anywhere else. Every browser since has adopted it, and every later web feature — frames, XMLHttpRequest, `fetch()`, canvas, service workers — has had to answer the question “how does this respect the same-origin policy?”

The same-origin policy is not a feature you turn on. It is the default state of the web, and every cross-origin capability is an exception carved out of it.

Who enforces it, and against whom

Three facts about the same-origin policy explain almost every surprise you will meet with CORS:

1 · The browser enforces it

Not the server, not the network. A server cannot enforce the SOP, and tools like `curl`, Postman or a Node script ignore it entirely. This is why “it works in Postman” tells you nothing about CORS.

2 · It protects the user

The SOP protects *the person at the keyboard* from hostile pages misusing their signed-in sessions. It does not protect your server from anyone who can send HTTP requests directly.

3 · It is about reading

Browsers still *send* many cross-origin requests — images, forms, links. What the policy withholds is the ability of a script to *read* the response. Chapter 4 makes this precise.

Why this matters in 2026 more than in 1996

In the 1990s a website was mostly one server rendering HTML. Today a typical product is a constellation: a single-page app on one origin, APIs on others, an authentication provider, a CDN, analytics, payment widgets. Legitimate cross-origin reads are everywhere, which means exceptions to the same-origin policy are everywhere — and each one is a small security decision written in a response header.

The stakes. When a CORS exception is written too broadly, the same-origin policy stops protecting the users of that API. Nothing crashes and no alarm sounds. The wall simply has a door in it that nobody meant to install.

LAUNCH NIGHT · 12:24 A.M.

“So the error isn't the bug,” you say slowly. “The error is the wall working.”

Ren nods. “Exactly. Our job is to open a door for our app — and only for our app.”

What exactly is an origin?

Three parts of a URL decide which side of the wall a page lives on. Get this definition exactly right and half of CORS debugging disappears.

origin = the exact triple (scheme, host, port), written `https://app.harborpay.example`

https : //app.harborpay.example : 443 /v1/balance?acct=7#top

SCHEME HOST PORT NOT PART OF THE ORIGIN

(implied by https) path, query, fragment are ignored

Figure 3.1 — Anatomy of an origin. The default port for https is 443 and for http is 80; when the port is the default it is omitted from the written form.

Same origin, or not? Compared with `https://app.harborpay.example`

URL	SAME?	WHY
<code>https://app.harborpay.example/settings</code>	SAME	Only the path differs
<code>https://app.harborpay.example:443/</code>	SAME	443 is the default port for https
<code>http://app.harborpay.example/</code>	DIFFERENT	Scheme differs: http is not https
<code>https://api.harborpay.example/</code>	DIFFERENT	Host differs, even though the parent domain matches
<code>https://harborpay.example/</code>	DIFFERENT	Host differs: no subdomain is a different host
<code>https://app.harborpay.example:8443/</code>	DIFFERENT	Port differs
<code>http://localhost:8080</code> vs <code>:8081</code>	DIFFERENT	Port differs — the basis of every lab in this book

The most common surprise. Your app and your API on sibling subdomains of the same company domain are **different origins**. The same-origin policy does not care who owns them. That is exactly Harbor Pay's launch-night situation.

Origin is not “site”

Browsers use a second, coarser boundary called a **site**: the scheme plus the *registrable domain* — the part of the host just below a public suffix such as `.com` or `.co.uk`. The list of public suffixes is maintained as the **Public Suffix List**, which is why `a.github.io` and `b.github.io` are different sites even though they share a parent domain.

TWO URLS	SAME ORIGIN?	SAME SITE?	WHICH RULES USE IT
<code>app.harborpay.example</code> <code>api.harborpay.example</code>	No	Yes	CORS uses origin. Cookie <code>SameSite</code> uses site, so these two are “same-site”.
<code>localhost:8080</code> <code>localhost:8081</code>	No	Yes	Ports never change the site. The labs rely on this.
<code>harborpay.example</code> <code>other.example</code>	No	No	Different in every sense.

Keep the two words apart. **CORS is about origins**. Cookie rules such as `SameSite` (Chapter 9) are about sites. Mixing them up produces bugs that appear in one environment and vanish in another.

The strange case of `null`

Some documents have no meaningful origin, and browsers give them an **opaque origin**, which is serialized in headers as the literal string `null`. You will see `Origin: null` from:

- pages opened from the local disk (`file://` URLs);
- documents created from `data:` URLs;
- sandboxed iframes (the `sandbox` attribute without `allow-same-origin`).

An opaque origin is never equal to any other origin, including another `null`. That makes `null` a *description*, not an identity — many unrelated documents all share the same string. Chapter 13 explains why a server must never treat it as a trusted caller.

How the browser tells the server

On cross-origin requests the browser adds an `Origin` request header naming the origin of the page that made the request. A page's script cannot forge it: it is one of the *forbidden request headers* that only the browser may set.

```
GET /v1/balance HTTP/1.1
Host: api.harborpay.example
Origin: https://app.harborpay.example    <- set by the browser, not by the page
Cookie: hp_session=...
```

Trust with care. The `Origin` header is reliable *inside a browser*; anyone using `curl` can type any value. Fine for CORS, which protects browser users — but never a substitute for authentication.

What the wall blocks, and what it doesn't

The same-origin policy is famous and widely misunderstood. It does not stop your browser from talking to other origins. It stops pages from *reading* what comes back.

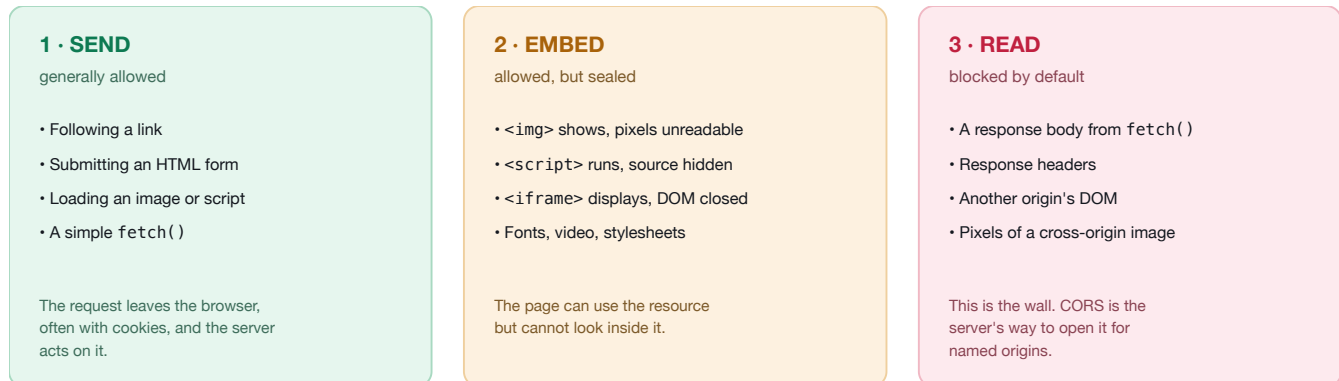


Figure 4.1 — Three kinds of cross-origin interaction. Only the third is blocked by default, and only the third is what CORS controls.

Why sending was never blocked

The web was built on links and forms that point at other sites. Blocking cross-origin *sending* would have broken the web of 1996, so it was never on the table. Instead, the rule became: **you may cause a request; you may not read the result** unless the other origin agrees. That compromise has two consequences you will meet again:

- **Side effects still happen.** If a cross-origin request changes data on the server, it changes it whether or not the page can read the response. That is the root of cross-site request forgery, and the reason Chapter 16 insists CORS is not a CSRF defense.
- **Embedding can leak.** Anything a browser will *execute* across origins — such as a script — must never contain private data. Chapter 5 shows what happened when sites forgot.

Remember this sentence. The same-origin policy governs reading, not sending. CORS lets a server say “this origin may read my response” — nothing more, nothing less.

Life before CORS

For a decade, developers who needed cross-origin data had no official way to get it. So they improvised — and every improvisation taught the industry something about what a safe design would need.

- **mid-1990s** JavaScript arrives in Netscape Navigator 2; the same-origin policy follows almost immediately.
- **1999–2005** XMLHttpRequest spreads and “Ajax” gets its name in 2005 — but it is same-origin only.
- **2005–2010** JSONP, server-side proxies and `document.domain` become the everyday workarounds.
- **2006–2007** Researchers show that private data served as executable JavaScript can be read by other sites (“JavaScript hijacking”).
- **2009–2014** Browsers implement CORS; it becomes a W3C Recommendation in January 2014 and later moves into the WHATWG Fetch Standard.

Workaround 1: JSONP

Scripts are allowed to load across origins (Figure 4.1, column 2). JSONP exploited that: instead of returning data, the server returned a *script* that called a function the page had defined, passing the data as its argument.

```
// the page defines a function, then adds a script tag pointing at the other origin
function showRates(data) { render(data); }
<script src="https://api.example/rates?callback=showRates"></script>

// the API responds with executable code rather than JSON
showRates({"USD": 1, "EUR": 0.92});
```

It worked everywhere, and it had two fatal properties. The page had to **run code chosen by the other server** with full privileges — total trust in the API. And the server had **no way to choose who could read**: any website could include the same script tag. JSONP was only ever safe for data you would publish to the whole internet.

Workaround 2: the server-side proxy

The page calls its *own* origin, and that server fetches from the other origin on its behalf. Because the browser only ever talks to one origin, the same-origin policy never triggers. Proxies remain a perfectly good pattern today (Chapter 20 recommends a form of it), but they add latency, cost, and a server that must be careful not to become an open relay for anyone's requests.

Workaround 3: `document.domain`

Two pages on sibling subdomains could both set `document.domain` to their shared parent and be treated as same-origin. It widened the wall for every page on those subdomains, not just the ones that needed it. Browsers have been retiring it; Chromium stopped honoring it by default in 2023. Treat it as history.

Workaround 4: `postMessage`

HTML5's `window.postMessage` lets windows on different origins exchange messages deliberately. It is still the right tool for cooperating *documents*, such as an embedded payment widget, and it is safe when the receiver checks `event.origin` against an exact expected value.

The lesson of JavaScript hijacking

RESEARCH • 2006–2007

When private data was served as code

In January 2006, security researcher Jeremiah Grossman described how a webmail service's contact list, delivered to its own pages as JavaScript, could be read by a page on a completely different site: the browser happily *ran* the cross-origin script, and the hostile page found a way to observe the data as it executed. The provider fixed the issue.

In 2007, researchers at Fortify Software (Brian Chess, Yekaterina Tsipenyuk O'Neil and Jacob West) published *JavaScript Hijacking*, showing the same weakness across many popular Ajax frameworks of the day and naming the class of bug.

What it taught: the “embed” column of Figure 4.1 is not a safe place for secrets. Responses containing private data should be plain data (JSON), never executable script, and should be readable only through a mechanism the server controls. That mechanism became CORS.

What a real solution needed

REQUIREMENT	THE WORKAROUNDS	HOW CORS ANSWERS IT
The server decides who may read	JSONP: anyone can read	The server names allowed origins in a response header
The page runs no foreign code	JSONP: runs the API's script	Responses are data handed to <code>fetch()</code>
Old servers stay safe by default	—	No header means no access; nothing changes for servers that never opt in
Dangerous requests need consent first	—	The preflight asks before the browser sends anything new
Credentials are opt-in, twice	Always attached	The page and the server must both explicitly agree

LAUNCH NIGHT • 12:41 A.M.

Someone asks whether JSONP would be quicker. Ren replies with a link to a 2007 research paper and two words: “*please don't.*” You open a blank document and write a heading: **What CORS actually does**. It is time to learn the protocol properly.

How CORS Works

CORS is a short conversation in HTTP headers. The browser says who is asking; the server says who may read. For anything unusual, the browser asks permission first. This part walks through that conversation one header at a time, and ends with the launch-night bug fixed properly.

BROWSER

OPTIONS /v1/transfers
Origin: https://app...
Access-Control-Request-Method: PUT

"may I?"

"you may."

SERVER

204 No Content
Access-Control-Allow-Origin: https://app...
Access-Control-Allow-Methods: PUT

Enter CORS

CORS does not tear down the wall. It gives the *server* a way to hand the browser a permission slip: “responses from me may be read by this origin.”

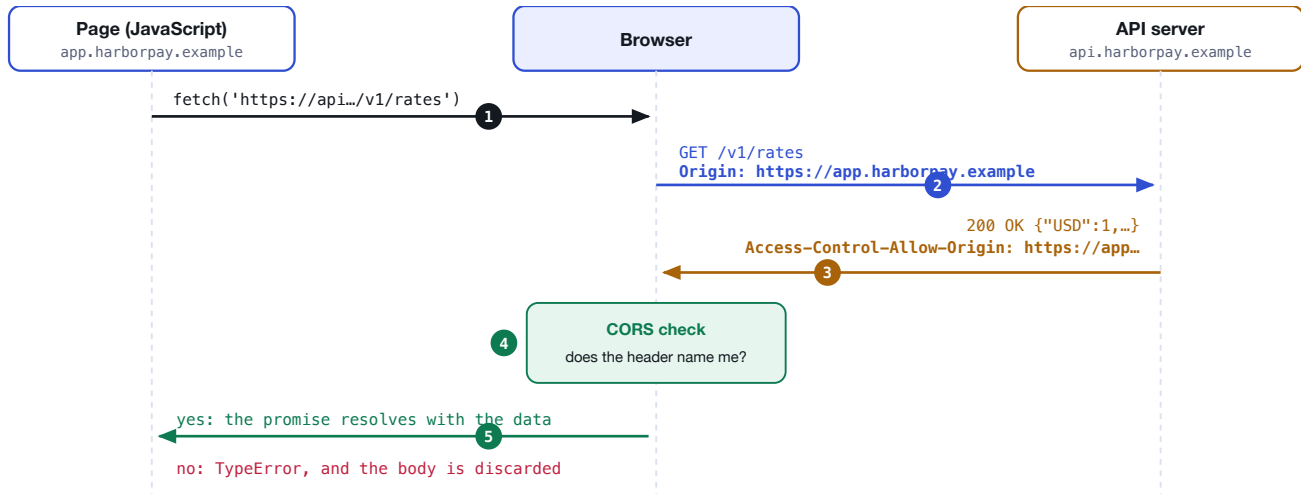


Figure 6.1 — The basic CORS exchange. Notice step 3: the server's response arrives at the browser *whether or not* it is allowed. The decision in step 4 is made by the browser, about whether to hand it to the page.

The core idea, in three sentences

1. On every cross-origin request, the browser sends an `Origin` header naming the page's origin.
2. The server answers normally, and adds `Access-Control-Allow-Origin` naming the origin it is willing to share with — or `*` for anyone.
3. The browser compares the two. If they match, the page may read the response; if not, the page gets an error and never sees the body.

Opt-in by design. A server that has never heard of CORS sends no `Access-Control-*` headers, so its responses stay unreadable to other origins — exactly as they were before CORS existed. Nothing old became less safe when CORS arrived.

The whole vocabulary on one page

Three headers are sent by the browser, six by the server. Every chapter in this part is about some subset of this table.

HEADER	SENT BY	MEANING
Origin	Browser	The origin of the page making the request. Always present on cross-origin requests.
Access-Control-Request-Method	Browser	In a preflight: the method the real request will use.
Access-Control-Request-Headers	Browser	In a preflight: the non-simple headers the real request will carry.
Access-Control-Allow-Origin	Server	The one origin allowed to read this response, or <code>*</code> . The heart of CORS.
Access-Control-Allow-Credentials	Server	<code>true</code> if a request with cookies may be read. Only valid with an exact origin.
Access-Control-Allow-Methods	Server	In a preflight response: methods the real request may use.
Access-Control-Allow-Headers	Server	In a preflight response: request headers the real request may carry.
Access-Control-Max-Age	Server	In a preflight response: how many seconds the answer may be cached.
Access-Control-Expose-Headers	Server	Response headers, beyond a small safe set, that the page's script may read.

Where CORS lives today

CORS became a W3C Recommendation in January 2014; in 2017 the W3C marked it obsolete because the living definition had moved into the **WHATWG Fetch Standard**, today's authority for every edge case. The origin itself is defined in **RFC 6454** (2011).

Three myths to drop now

“CORS protects my API.”

It protects *browser users* of your API. Scripts and command-line tools ignore it; your API still needs real authorization.

“A CORS error means the request failed.”

Often the request reached the server and succeeded. The browser only refused to show the page the response. Lab 3 proves it.

“I fix CORS in the front end.”

Permission comes from the server's response headers. No option in `fetch()` can grant a page access the server didn't give.

Simple requests

Some cross-origin requests go straight to the server with no questions asked. The reason is historical, and once you see it, the rules stop looking arbitrary.

The logic: nothing a form couldn't already do

Long before CORS, any page could build an HTML form that submitted a `GET` or a `POST` to any server, with a handful of content types. Servers already had to live with those requests arriving from anywhere. So CORS reasoned: if a script sends a request that a form *could have sent*, letting it through creates no new danger. Such a request is called **simple** (the Fetch Standard's term is a request that needs no *CORS-preflight*).

What makes a request simple

PART	ALLOWED VALUES	NOTES
Method	GET, HEAD, POST	Anything else (PUT, PATCH, DELETE...) needs a preflight.
Headers set by the page	Accept, Accept-Language, Content-Language, Content-Type	The CORS-safelisted request headers (plus a simple <code>Range</code>). Any other header, such as <code>Authorization</code> or <code>X-Request-Id</code> , needs a preflight.
Content-Type	application/x-www-form-urlencoded multipart/form-data text/plain	The three types a form can send. <code>application/json</code> is <i>not</i> on the list.

There are a few rarer conditions too (for example, safelisted header values have length limits, and upload progress listeners force a preflight). Browsers apply them for you; the three rows above cover almost every real case.

Simple or not? Five fetch calls

CALL	RESULT
<code>fetch(url)</code>	SIMPLE a plain GET
<code>fetch(url, {method: 'POST', body: formData})</code>	SIMPLE multipart/form-data
<code>fetch(url, {method: 'POST', headers: {'Content-Type': 'application/json'}, ...})</code>	PREFLIGHT JSON content type
<code>fetch(url, {headers: {'Authorization': 'Bearer ...'}})</code>	PREFLIGHT non-safelisted header
<code>fetch(url, {method: 'DELETE'})</code>	PREFLIGHT non-simple method

The consequence to remember. A simple request is **sent before any permission is checked**. If it changes data on the server, the change happens even when the browser then refuses to show the response. CORS decides who may *read*; it was never designed to decide who may *write* with a form-like request. Chapter 16 turns this into concrete defenses.

Designing with simple requests in mind

Knowing the rule lets you choose deliberately. Read-only public endpoints can stay simple and fast. Endpoints that change data should *require* something a form cannot send — a JSON body or a custom header — so that browsers must preflight them, and your policy gets a say before anything happens. The lab server's notes endpoint does exactly this: it refuses any request that is not `application/json`.

LAUNCH NIGHT · 1:05 A.M.

You check the dashboard's code. The balance call is a plain `GET` — a simple request. The browser sent it, the API answered with the balance, and the browser threw the answer away because no `Access-Control-Allow-Origin` came back. “So our API has been doing the work all night,” you murmur, “and the browser has been shredding it.”

The preflight

For any request a form couldn't send, the browser asks first. The question is an `OPTIONS` request, and nothing else is sent until the server says yes.

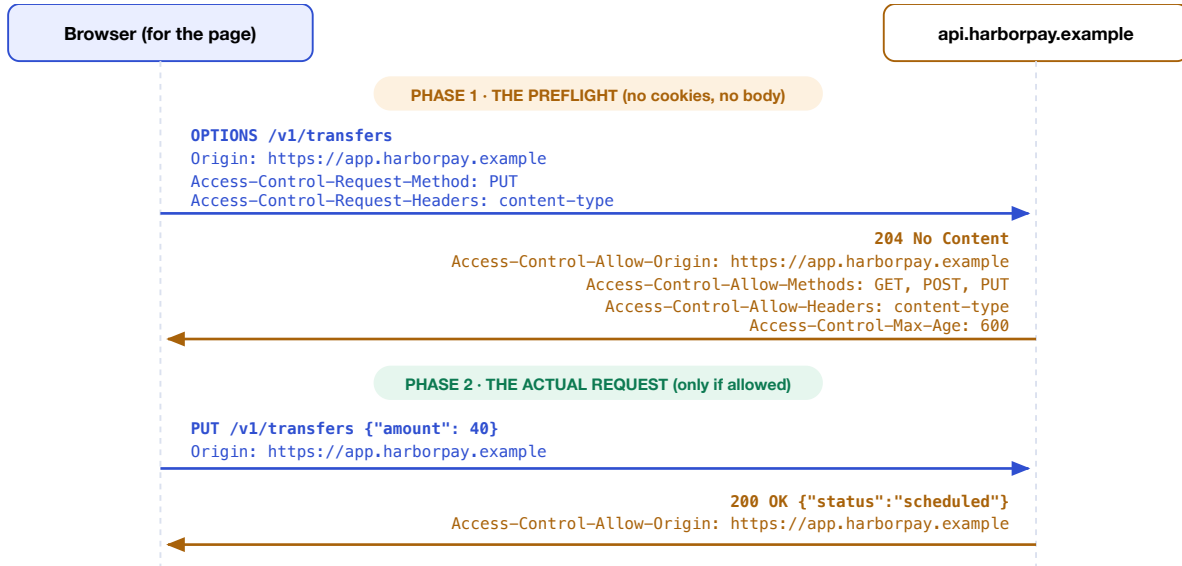


Figure 8.1 — A prefighted request. If the phase-1 answer does not grant the method, every header, and the origin, phase 2 is never sent and the page receives a network error.

What the browser checks in the preflight answer

- The status is a success (2xx). A redirect or a 401 fails the preflight.
- `Access-Control-Allow-Origin` matches the page's origin (or is `*` when no credentials are involved).
- The requested method appears in `Access-Control-Allow-Methods`. Simple methods are always allowed.
- Every header in `Access-Control-Request-Headers` appears in `Access-Control-Allow-Headers`.

Why the preflight carries no credentials

The preflight is sent **without cookies** and without a body, even when the real request will include them. It is a question about *policy* (“would you accept this kind of request from this origin?”), not about any user. Two practical consequences follow:

- **Don't require authentication on OPTIONS**. An API gateway that answers unauthenticated preflights with 401 breaks every non-simple request from the browser. Answer the preflight from policy alone, then authenticate the actual request.
- **A preflight never performs the action**. Your handler for the real `PUT` should not run for an `OPTIONS` request. Most frameworks route preflights to CORS middleware automatically; hand-written servers must do it themselves, as the lab server's `applyCors` function does.

Wildcards in a preflight answer

The Fetch Standard allows `*` in `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers`, meaning “any” — but only for requests **without credentials**. With credentials, `*` is treated as a literal name that matches nothing useful. One more exception: the wildcard in `Allow-Headers` never covers `Authorization`, which must always be listed by name. Listing exactly what you use is clearer anyway, and it documents your API's contract.

Caching the answer

A preflight costs a full round trip before every non-simple request. `Access-Control-Max-Age` lets the browser remember a successful answer, keyed on the origin, the URL, and the method or header being allowed.

SITUATION	EFFECTIVE CACHE LIFETIME
No <code>Max-Age</code> header	5 seconds (the Fetch Standard's default)
<code>Max-Age: 600</code>	10 minutes
<code>Max-Age: 86400</code>	Capped by the browser: Chromium limits it to 2 hours, Firefox to 24 hours

A value of a few minutes to an hour is a sensible default. Long caches mean a policy change takes that long to reach browsers that already asked; short ones cost latency. Lab 7 lets you count preflights as you change the value.

Performance tip. Preflights are cached per URL, so an API with identifiers in its paths pays for a fresh preflight on every new URL. Serving the API from the app's own origin (Chapter 20) removes the cost entirely.

LAUNCH NIGHT • 1:20 A.M.

The transfer screen will be worse than the balance card: it sends JSON with `PUT`, so the browser won't even send it until a preflight succeeds. You add `PUT` and `content-type` to your notes.

Credentials and cookies

Reading public data across origins is easy. Reading data that belongs to a signed-in user is where CORS gets strict — deliberately, because that is where the danger is.

Two opt-ins, from two different parties

By default, `fetch()` sends cookies only to the page's own origin. For a cross-origin request to carry the user's cookies *and* have its response readable, both sides must opt in:

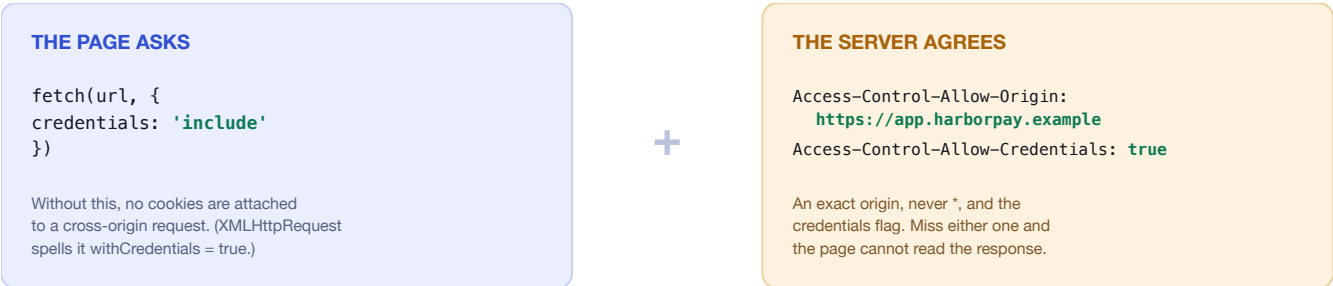


Figure 9.1 — Credentialed CORS needs consent from both ends. The page must ask for cookies; the server must name the exact origin and say `true`.

The rules with credentials

HEADER	WITH CREDENTIALS
Access-Control-Allow-Origin: *	REJECTED must be the exact origin
Access-Control-Allow-Credentials	REQUIRED exactly the string <code>true</code>
Allow-Methods / Allow-Headers: *	NO WILDCARD list the names
Access-Control-Expose-Headers: *	NO WILDCARD list the names

Why “*” and credentials may never meet

A `*` says “this response is fit for anyone.” That is safe only for a response that is the same for everyone. A response computed from a user's cookies is, by definition, not for anyone — it is for that user, in pages that the server trusts. The browser enforces the distinction so that a single careless header cannot publish every signed-in user's data to every website. Chapter 13 shows the tempting workaround that defeats this protection, and why it must be avoided.

Cookies have rules of their own

CORS decides whether a page may read a credentialed response. Whether the cookie is *attached* in the first place is decided separately, by cookie attributes and browser privacy settings:

- **SameSite** limits cookies on cross-site requests (Chapter 3's “site,” not origin). **Strict** never sends them cross-site, **Lax** sends them only on top-level navigations such as following a link, and **None** sends them everywhere but requires **Secure**. Chrome treats a cookie without the attribute as **Lax**.
- **Third-party cookie restrictions** in Safari and Firefox block or partition cookies for cross-site requests by default, and other browsers let users choose. If your app and API are on unrelated sites, credentialed CORS may work in one browser and silently lose its cookie in another.

Harbor Pay is lucky: `app.harborpay.example` and `api.harborpay.example` are the **same site**, so a `SameSite=Lax` session cookie is attached to the dashboard's credentialed requests. If your front end and API live on different registrable domains, prefer serving the API from the app's own origin (Chapter 20), or use tokens in an `Authorization` header instead of cookies.

LAUNCH NIGHT • 1:48 A.M.

You write the fix. It is not a one-liner, and that is the point. The API gets an explicit list containing a single origin, `https://app.harborpay.example`. When a request's `Origin` is on that list, the server names it exactly, adds `Access-Control-Allow-Credentials: true`, and answers preflights with the three methods and one header the dashboard actually uses. Every other origin gets no CORS headers at all.

Ren reads the diff twice. “*Ship it.*” At 2:03 a.m. the balance card fills in. Nobody cheers; everyone is too tired. But you notice Ren adding a ticket to the backlog titled **Audit every CORS policy we have**.

```
// The launch-night fix, in outline (the lab server implements it in full)
const ALLOWED = ['https://app.harborpay.example'];

if (ALLOWED.includes(req.headers.origin)) {           // exact match, nothing clever
  res.setHeader('Access-Control-Allow-Origin', req.headers.origin);
  res.setHeader('Access-Control-Allow-Credentials', 'true');
}
res.setHeader('Vary', 'Origin');                      // Chapter 17 explains why
```

Why this is safe. The server only ever repeats an origin it has already decided to trust, compared as a whole string against a list someone wrote down and reviewed. There is no pattern to get wrong and no way for a caller to talk its way onto the list.

Reading the response

Even when CORS allows a response, the page does not see all of it. And when CORS refuses, the page is told almost nothing — on purpose.

Only some headers are visible

A script reading a cross-origin response can see the body and the status, but only a small set of **CORS-safelisted response headers**:

Cache-Control · Content-Language · Content-Length · Content-Type · Expires · Last-Modified · Pragma

Anything else — a request ID, a rate-limit counter, a pagination link — is hidden from `res.headers.get()` unless the server lists it:

Access-Control-Expose-Headers: X-Request-Id, X-RateLimit-Remaining, Link

DevTools will still show the header in the Network tab, which makes this one of the most confusing CORS behaviors: you can see it, but your code gets `null`. Lab 5 reproduces exactly that.

Opaque responses and mode: 'no-cors'

Developers fighting a CORS error sometimes discover `fetch(url, {mode: 'no-cors'})` and think the problem is solved, because the error disappears. It isn't. A no-cors request returns an **opaque response**: status `0`, no headers, and an empty body. The browser may *use* it (a service worker can cache it for an image tag), but your script can never read it. `no-cors` is how the browser keeps old embedding behavior working — not a way around the same-origin policy.

Why the error says so little

When a CORS check fails, `fetch()` rejects with a generic error — `TypeError: Failed to fetch` in Chromium, `NetworkError when attempting to fetch resource` in Firefox, `Load failed` in Safari. The script cannot learn the status code, whether the server was reachable, or which header was wrong.

That vagueness is a security feature. If a page could distinguish “server said 404” from “server said 200 but CORS refused,” it could probe other origins — including servers on your private network — and learn about them without ever being allowed to read them. The detailed explanation is written only to the DevTools console, for the human at the keyboard.

Debugging rule of thumb. Never debug CORS from your application's error handler. Open the Console for the reason and the Network tab for the raw headers; the next chapter shows how to read them.

Debugging CORS like a pro

Every CORS failure is one of a handful of mismatches between what the browser asked and what the server said. Find which, and the fix is usually a single header.

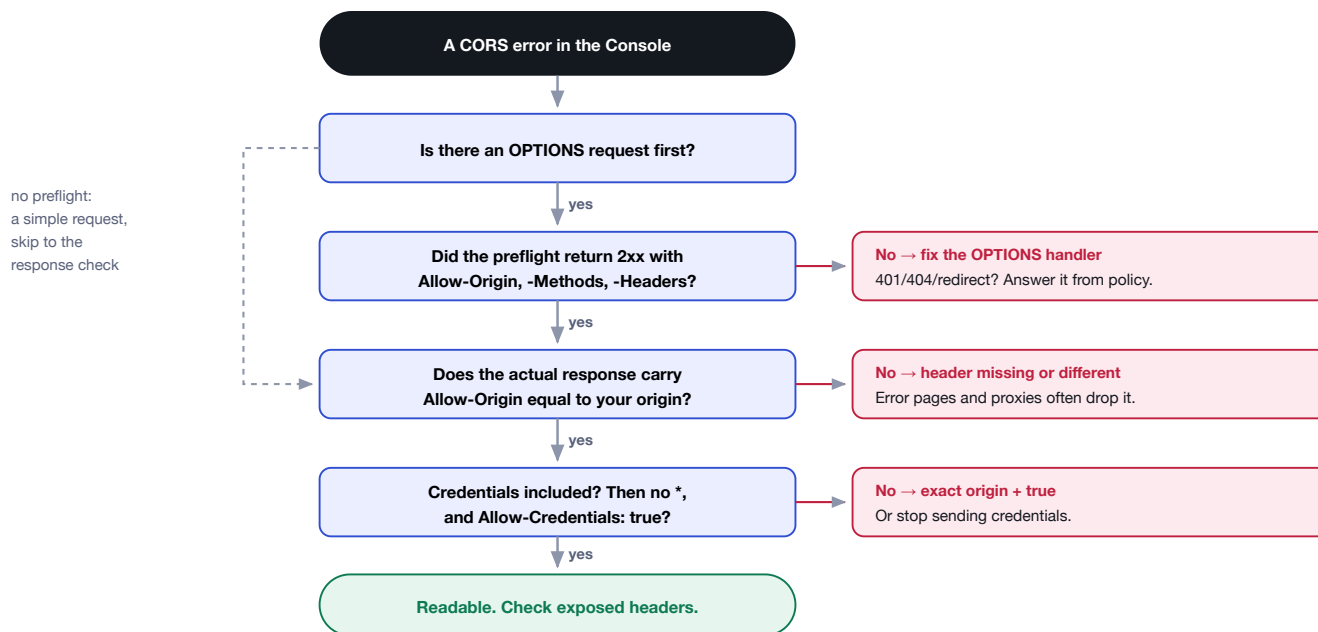


Figure 11.1 — A debugging path for any CORS error. Work top to bottom in the Network tab; the first “no” is your bug.

Tools that tell the truth

- **Console:** the browser's exact reason, in plain language (next page).
- **Network tab:** filter by the API's host; look for the `OPTIONS` row and compare request and response headers side by side.
- **curl:** replay what the browser sent, to see exactly what the server returns. It ignores CORS itself, so use it to inspect headers, never to conclude “it works.”

The Console messages, decoded

Chromium's wording, abbreviated; Firefox and Safari phrase the same conditions differently.

THE MESSAGE SAYS...	WHAT IT MEANS, AND THE USUAL FIX
No 'Access-Control-Allow-Origin' header is present	The server didn't opt in — or an error page, proxy or CDN stripped the header. Check the actual response, including 4xx/5xx ones.
...has a value 'X' that is not equal to the supplied origin	The server named a different origin: a typo, a missing port, <code>http</code> vs <code>https</code> , or a cached response for another origin (Chapter 17).
...must not be the wildcard '*' when the request's credentials mode is 'include'	Name the exact origin and add <code>Allow-Credentials: true</code> , or stop sending credentials if the data is public.
...'Access-Control-Allow-Credentials' header... must be 'true'	The page sent credentials; the server didn't agree. Add the header for trusted origins only.
Response to preflight request... does not have HTTP ok status	The <code>OPTIONS</code> request hit authentication, a 404, or a redirect. Answer preflights before auth.
Request header field X is not allowed by Access-Control-Allow-Headers	Add the header name to <code>Allow-Headers</code> — or stop sending it if you don't need it.
Method X is not allowed by Access-Control-Allow-Methods	Add the method to <code>Allow-Methods</code> for the routes that accept it.

Inspecting headers with curl

Replay a preflight exactly as the browser would send it, and read what comes back:

```
curl -i -X OPTIONS https://api.harborpay.example/v1/transfers \
  -H 'Origin: https://app.harborpay.example' \
  -H 'Access-Control-Request-Method: PUT' \
  -H 'Access-Control-Request-Headers: content-type'
```

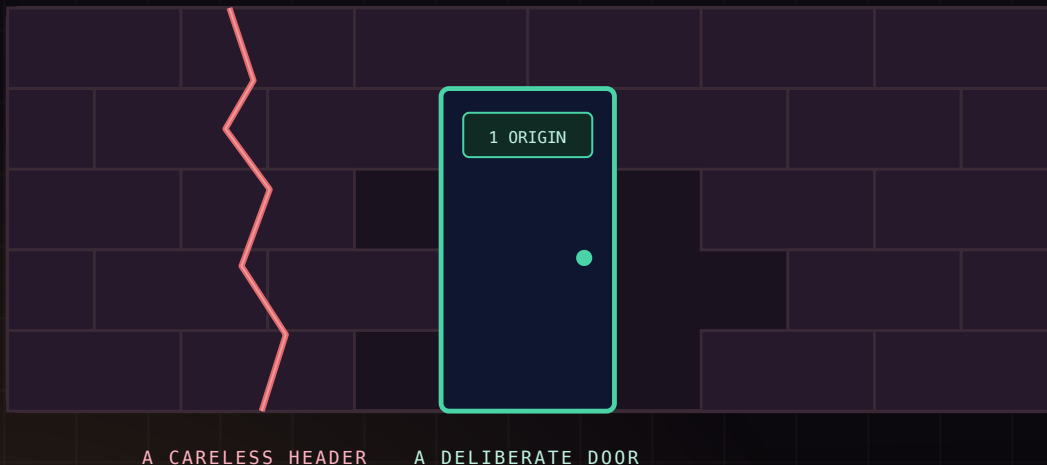
Repeat it with an origin that should *not* be allowed: a correct policy answers only the first with permission headers. `node server.js check` prints this comparison.

Four traps that waste an afternoon

- **Only the error path fails.** The framework adds CORS headers on success but not on the 500 page, so real errors look like CORS errors.
- **Two layers both add headers.** The app and the proxy each send `Allow-Origin`; two values are invalid and the browser rejects both.
- **A redirect in the middle.** `http` → `https` or a missing trailing slash redirects the preflight, which fails.
- **A trailing slash in the allowlist.** `https://app.example/` is not an origin; `https://app.example` is.

When the Wall Has Holes

A CORS policy is a security decision written as a response header. Written well, it opens one precise door. Written carelessly, it quietly switches off the browser's protection for every user of your API. This part explains why that matters, the mistakes behind real incidents, the attack patterns they enable, and one thing CORS was never meant to do.



Why CORS is a security priority

A CORS policy is often the only thing standing between a page your users happen to visit and the data your API holds about them. That makes a single response header one of the most consequential lines in your codebase.

THREE WEEKS LATER • SECURITY REVIEW, 10:00 A.M.

Ren's backlog ticket has become a meeting. On the screen: a spreadsheet of every Harbor Pay service and the CORS headers it returns. Forty-one rows, some from teams that no longer exist.

“Before we look at any row,” Ren says, “let's agree on why this is worth a morning.”

Six reasons it deserves attention

1 • It guards the user's own session

Your authentication works perfectly and still doesn't help: the request *really is* from the signed-in user's browser, carrying their real cookie. The only question is which page gets to read the answer. That question is CORS.

2 • APIs are the product now

Single-page apps, mobile back ends and partner integrations put account data, payment details and personal records behind JSON endpoints. Those are exactly the responses CORS decides about.

3 • Mistakes are silent

A too-strict policy produces red errors and angry tickets. A too-loose policy produces nothing at all — every test passes. The feedback loop rewards loosening and never punishes it.

4 • The fix is tempting and tiny

Under launch pressure, the difference between a correct policy and a dangerous one is a few characters. Forums are full of snippets that make errors disappear by trusting everyone.

5 • Reach is multiplied

A policy bug is not one user's problem. It affects every signed-in user of that API, from every browser, until someone notices.

6 • Standards bodies track it

The weakness has its own entries — CWE-942 (*Permissive Cross-domain Policy with Untrusted Domains*) and CWE-346 (*Origin Validation Error*) — and the OWASP Web Security Testing Guide has a dedicated test, WSTG-CLNT-07.

What is actually at stake

If a policy lets an untrusted origin read *credentialed* responses, any page a signed-in customer visits can see what that customer sees in your app, through their own session. No password is stolen and no server is breached, so the logs show only normal requests.

The one-line summary. CORS misconfiguration turns your users' browsers against them. It is a privacy and data-exposure problem first, and its blast radius is every signed-in user.

Where risk actually lives

Not every permissive header is a problem. Risk depends on two questions: **does the response depend on who is asking** (cookies, network location, or other ambient credentials), and **whom does the policy allow to read it?**

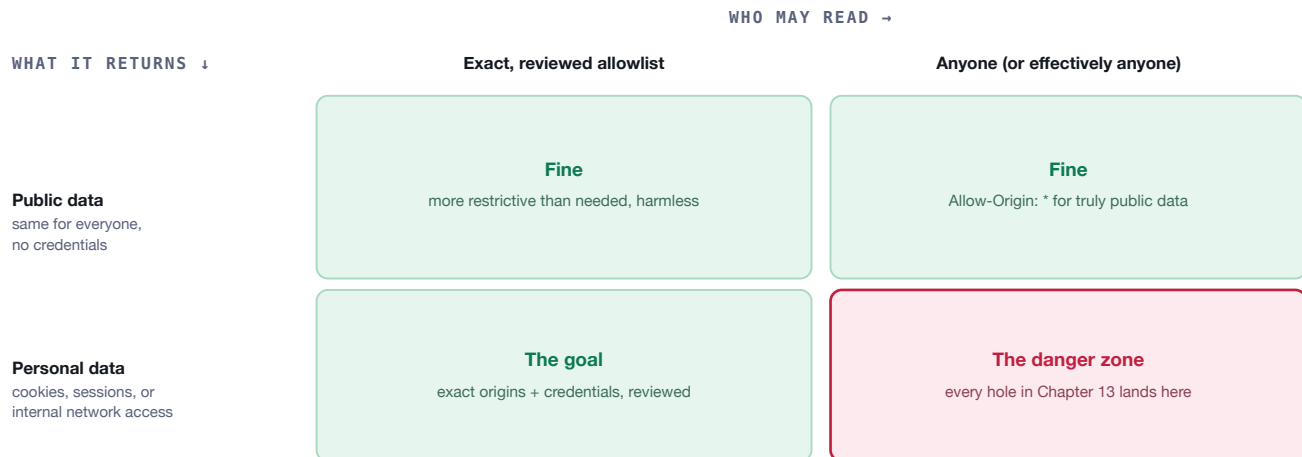


Figure 12.1 — Where CORS risk lives. Only one quadrant is dangerous, and every mistake in the next chapter is a way of landing in it by accident.

“But we have authentication”

This is the most common objection in a review, and it misses the point. Authentication answers “is this request from Alice?” In the danger zone the answer is honestly yes — Alice's browser sent it, with Alice's cookie. What went wrong is that a page Alice didn't intend to trust got to read the reply. Token-based APIs that never use cookies are far less exposed for the same reason: the browser does not attach a bearer token automatically, so there is no ambient credential to misuse.

What a good review asks

1. Which responses from this service depend on cookies, sessions or network location?
2. For those responses, exactly which origins can read them, and who approved each one?
3. How is the `Origin` header compared — whole-string equality, or something cleverer (and therefore riskier)?
4. Are the pages on every trusted origin themselves well defended against script injection?
5. Do all responses, including errors and cached ones, carry the right headers and `Vary: Origin`?

SECURITY REVIEW • 10:25 A.M.

The team sorts the spreadsheet by the first question. Nineteen services return nothing personal; they can relax. Twenty-two do. Ren highlights those rows in yellow. “Now,” Ren says, “let's see how each one decides who's allowed in.”

Configuration mistakes that matter

Almost every serious CORS problem is one of six patterns. Each one looks reasonable in a code review, and each one has a simple, boring fix.

MISTAKE 1 · ECHOING ANY ORIGIN, WITH CREDENTIALS

“Copy back whatever Origin arrives”

The server reads the request's `Origin` and repeats it in `Access-Control-Allow-Origin`, together with `Allow-Credentials: true`. It always “works,” which is why it spreads. It is also exactly the combination the browser's `*`-with-credentials rule exists to prevent: every origin on the internet is now trusted with your users' sessions. Watch for framework settings with friendly names — in some popular middleware, setting the origin option to `true` means “reflect the request's origin.”

Fix: compare the origin against an explicit list and repeat it only on an exact match. If the data is truly public, use `*` without credentials.

MISTAKE 2 · LOOSE MATCHING

Prefixes, suffixes, substrings and hand-written patterns

Checks like “the origin *contains* our domain,” “*ends with* our domain,” or a regular expression with an unescaped dot or a missing anchor all accept more than their authors intended — including hostnames that someone else can register. The researchers in Chapter 15 found this class of bug to be widespread.

Fix: don't pattern-match origins. Keep a list of complete origin strings (scheme, host and port) and use exact equality. If you must support many subdomains, parse the origin with a URL parser and compare the parsed hostname against an exact set.

MISTAKE 3 · TRUSTING NULL

Allowing Origin: `null` “for local testing”

Developers add `null` to an allowlist so that a page opened from disk can call the API. But `null` is shared by every sandboxed iframe, every `data:` document and every local file on every machine (Chapter 3). Trusting it trusts all of them at once.

Fix: never allowlist `null`. Serve test pages from a real local origin such as `http://localhost:8080`, and keep development origins out of production configuration.

MISTAKE 4 · TRUSTING PLAIN HTTP**Allowing `http://app.harborpay.example` as well as `https`**

An `http` origin is only as trustworthy as every network between the user and the server. On a hostile Wi-Fi network, content served over plain HTTP can be altered in transit, so trusting the `http` version of your own site extends your API's trust to whoever controls that network.

Fix: allow only `https` origins in production, and use HSTS so that your sites are never loaded over plain HTTP in the first place.

MISTAKE 5 · TRUSTING EVERY SUBDOMAIN**“Anything under `harborpay.example` is ours”**

A company domain accumulates marketing microsites, old campaign pages, third-party-hosted help centers and forgotten staging servers. Trusting all of them means your most sensitive API is only as safe as your least-maintained subdomain — including any with a cross-site scripting bug or a dangling DNS record pointing at a service someone else can claim.

Fix: list the specific origins that need access. Treat adding one as a security decision, and remove entries when the site that needed them is retired.

MISTAKE 6 · NETWORK LOCATION AS A CREDENTIAL**Allow-Origin: * on an internal API**

“It's only reachable from the office network, so * is fine.” But the code that makes the request runs *inside an employee's browser*, which is on the office network. A wildcard on an unauthenticated intranet API lets any page an employee visits read it. Local services on `localhost` have the same problem (Chapter 18).

Fix: require real authentication on internal services, and never combine “trusted because of where it is” with *.

Plus one that breaks things before it breaks security

A dynamic policy (one that names different origins for different requests) without `Vary: Origin` lets caches serve one origin's permission to another. Chapter 17 covers it in depth; the fix is one header, on every response.

The pattern behind all six. Every mistake here replaces a **short, explicit list of trusted origins** with a rule that trusts a category — “whoever asks,” “anything that looks like us,” “anything local.” Categories grow without anyone deciding. Lists only grow when someone does.

SECURITY REVIEW · 11:40 A.M.

Of the twenty-two yellow rows, fourteen use exact lists. Five check whether the origin *ends with* the company domain; two trust `null` for an old test harness; and one forgotten reporting service echoes every origin with credentials. Ren opens eight tickets and marks the last one urgent.

Attack patterns, drawn

You cannot defend against a shape you have never seen. This chapter draws the seven patterns behind real cross-origin incidents, so you can recognize them in a design review, spot their traces in your logs, and know exactly where to break the chain.

SECURITY REVIEW • 1:30 P.M.

After lunch Ren stands at the whiteboard. “Tickets fix today's bugs,” Ren says. “Patterns stop next year's. Every cross-origin attack I've seen is one of seven shapes.” Ren draws the first one.

How to read the diagrams

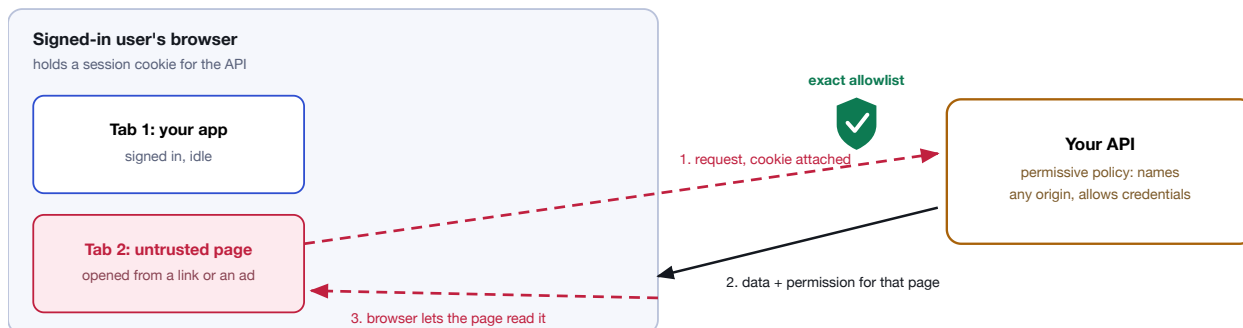
Red dashed arrows are steps an attacker can cause or influence — usually just by getting a signed-in user to open a web page.

Solid arrows are the normal behavior of browsers and servers. Most attacks need nothing unusual from either.

Green shields mark where a defense breaks the chain. One shield anywhere on the path is enough.

PATTERN 1 • THE BORROWED SESSION

A hostile page reads your API through the user's own session



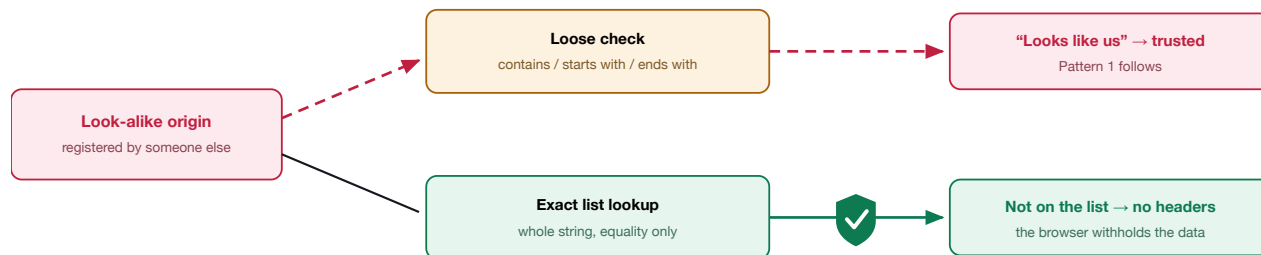
What happens. Nothing is stolen from the server and no password is guessed. The API does what it always does — answers the user's browser — and its CORS policy tells the browser that the untrusted page may read the answer.

Warning signs. Credentialed responses whose `Access-Control-Allow-Origin` names an origin you never approved; `Origin` values in your logs that no team recognizes.

Break the chain. An exact allowlist (Chapter 13), no credentials where the data isn't personal, and cookies with `SameSite` so the session isn't attached cross-site in the first place.

PATTERN 2 · THE LOOK-ALIKE

An origin that merely resembles yours passes a loose check



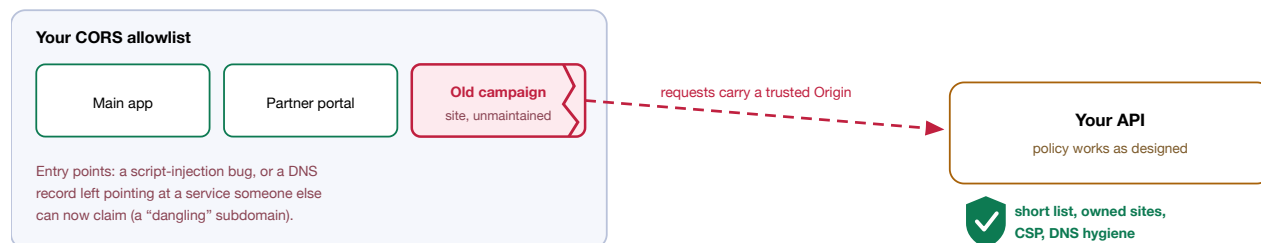
What happens. A string test written to recognize your domain also accepts names that contain or end with the same characters — names anyone can register.

Warning signs. Origin checks built from `includes`, `endsWith`, or hand-written regular expressions; allowlists described in words rather than listed.

Break the chain. Exact equality against complete origin strings, and tests that prove near-miss origins are refused (Chapter 20).

PATTERN 3 · THE COMPROMISED FRIEND

A trusted origin is taken over, and brings its trust with it



What happens. The policy is correct, but one of the origins it trusts is no longer trustworthy. Anything running there speaks with that origin's voice.

Warning signs. Allowlist entries nobody owns; subdomains whose DNS points at deleted cloud resources; old sites without a content security policy.

Break the chain. Trust only what you maintain, give every entry an owner, remove retired sites, and monitor DNS for dangling records.

PATTERN 4 · THE ANONYMOUS FRAME

Trusting null trusts everyone who can be anonymous



What happens. Many unrelated documents share the same `null` origin, and some of them can be created by any website. **Warning signs:** `null` in an allowlist or a test configuration that reached production. **Break the chain:** serve test pages from a real local origin and refuse `null` everywhere.

PATTERN 5 · THE BLIND WRITE

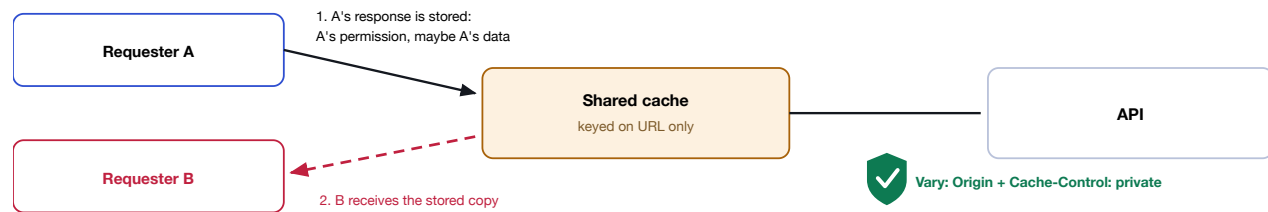
The response is hidden, but the action already happened



What happens. CORS only protects reading (Chapter 16), so a request a form could send changes data even though its answer is withheld. **Warning signs:** write endpoints that accept form encodings; state changes arriving with `Sec-Fetch-Site: cross-site`. **Break the chain:** `SameSite` cookies plus a server-side check on every write.

PATTERN 6 · THE CONFUSED CACHE

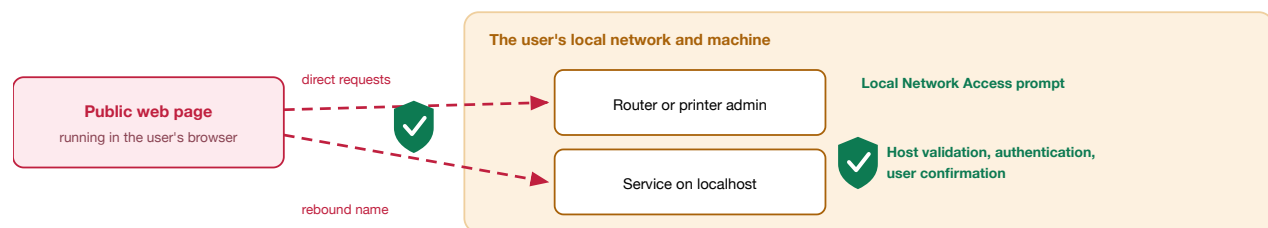
One requester's answer is served to another



What happens. A cache that doesn't know the response depends on who asked hands one requester's permission, or one user's data, to the next. **Warning signs:** CORS errors that come and go; personalized responses served with public cache headers. **Break the chain:** `Vary: Origin` on every dynamic CORS response, and private or no-store caching for anything personal (Chapter 17).

PATTERN 7 · THE INSIDE-OUT REQUEST

A public page reaches into the user's own network



What happens. Devices and helpers that trust “anyone on my network” receive requests from web pages, because the browser is on that network. **Warning signs:** local services with no authentication; requests with unexpected `Host` headers. **Break the chain:** the practices in Chapter 18.

One page, seven shapes

PATTERN	WHAT THE ATTACKER NEEDS	PRIMARY DEFENSE
1 Borrowed session	A permissive credentialed policy	Exact allowlist; <code>SameSite</code>
2 Look-alike	A loose origin check	Exact equality, tested
3 Compromised friend	A weak site on your allowlist	Short, owned list; CSP; DNS hygiene
4 Anonymous frame	<code>null</code> on your allowlist	Never trust <code>null</code>
5 Blind write	A write endpoint that accepts simple requests	CSRF defenses on every write
6 Confused cache	A cache without <code>Vary: Origin</code>	<code>Vary</code> ; private caching
7 Inside-out request	A local service that trusts its network	Host checks; authentication

Being prepared: detect, respond, learn

A prepared team knows what an attempt looks like in its logs and what to do in the first hour. Most of these signals are already in your access logs.

Signals worth alerting on

SIGNAL	WHERE TO LOOK	WHAT IT MAY MEAN
A credentialed response names an origin that isn't on your list	Response logging, API gateway	A reflecting or loosely matched policy (patterns 1, 2). Should be impossible; treat as an incident.
Unfamiliar <code>Origin</code> values on authenticated endpoints	Access logs	Someone probing your policy, or an integration nobody documented.
<code>Origin: null</code> on API traffic	Access logs	Sandboxed or local documents calling you (pattern 4).
State changes with <code>Sec-Fetch-Site: cross-site</code>	Access logs, WAF	Possible forged writes (pattern 5).
Unexpected <code>Host</code> headers on local or internal services	Service logs	Possible DNS rebinding (pattern 7).
Subdomains pointing at deleted cloud resources	DNS inventory	A trusted origin waiting to be claimed (pattern 3).

The first hour of a CORS incident

1. **Contain.** Replace the faulty rule with an exact allowlist, or temporarily remove `Allow-Credentials`, and deploy. This is usually a one-line configuration change.
2. **Purge.** Clear CDN and proxy caches that may hold responses with the old headers.
3. **Scope.** Search logs for credentialed requests whose `Origin` was not on the approved list: which origins, which endpoints, which users, over what period.
4. **Protect users.** If personal data may have been read, invalidate the affected sessions and follow your notification obligations.
5. **Prevent recurrence.** Add the offending origin pattern to your CORS test suite (Chapter 20) so the same mistake fails a build next time.

The preparation that matters most. An inventory of every service's CORS policy, an owner for every allowed origin, and automated tests that fail on anything broader. Teams that have these three rarely need the incident checklist above.

SECURITY REVIEW • 3:05 P.M.

Ren pins a photo of the whiteboard to the team channel: seven shapes, seven green shields. “Find the shape first,” Ren says. “The fix is usually already drawn.”

Case studies from the real world

These are public incidents and published research, summarized from the original disclosures. Each one is told for its lesson: what the system trusted, why that trust was misplaced, and what fixed it.

RESEARCH · OCTOBER 2016 · OWASP APPSEC USA

“Exploiting CORS Misconfigurations for Bitcoins and Bounties”

James Kettle of PortSwigger surveyed how real websites validated the `Origin` header and found that many used simple string operations — reflection, prefix and suffix checks — rather than exact comparison. Among the findings he described were cryptocurrency exchanges whose CORS policies could have let another website read sensitive account data from signed-in users; the issues were reported to the affected organizations. The talk also showed how CORS mistakes combine with other bugs, such as cache behavior and trusted-but-vulnerable subdomains.

Lesson: the way you *compare* origins is security-critical. The most common bugs were not exotic; they were convenient string checks (Chapter 13, mistakes 1 and 2).

RESEARCH · AUGUST 2018 · USENIX SECURITY SYMPOSIUM

“We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS”

Jianjun Chen and colleagues measured CORS deployments across a large set of popular websites. They found insecure misconfigurations to be common, identified weaknesses in how CORS interacts with other web mechanisms, and noted that many sites whose policies varied by origin failed to send `Vary: Origin`. The authors released an open-source scanner, CORScanner, and proposed clarifications, some of which were incorporated into the specification.

Lesson: CORS mistakes are not rare accidents but a systematic pattern, driven by an inexpressive policy language and poor developer understanding. Treat CORS as something to *test*, not *assume*.

WEB SECURITY HISTORY · OCTOBER 2005

The Samy worm

A cross-site scripting flaw on MySpace let a user named Samy Kamkar put a script in a profile page. Anyone who viewed the profile ran the script, which added Samy as a friend and copied itself to the viewer's own profile. It reached over a million profiles in about a day, and MySpace went offline to clean it up.

Lesson for CORS: the same-origin policy only separates origins; it cannot protect an origin from script running *inside* it. Every origin on your CORS allowlist extends its trust to your API, so each one must be defended against script injection — a bug on a trusted origin is a bug in your API's security.

VULNERABILITY • JULY 2019 • CVE-2019-13450

A video-conferencing app's localhost web server

Security researcher Jonathan Leitschuh disclosed that the Zoom client for macOS installed a local web server, listening on `localhost` port 19421, so that websites could launch meetings with one click. Any website could send it requests, and it could join a user to a meeting — with video on in some configurations — without meaningful confirmation. Because CORS prevented pages from reading the server's responses, the app reportedly signaled results through the dimensions of an image instead. The server also remained installed after the app was removed. Zoom shipped updates removing the server, and Apple pushed a silent macOS update to remove it from affected machines.

Lesson: a local service is reachable from every web page the user opens. CORS only limits *reading*; requests still arrive and actions still happen. Local services need explicit user confirmation and real authentication, not reliance on being “only on localhost” (Chapter 18).

VULNERABILITY • JANUARY 2018 • CVE-2018-5702

A BitTorrent client's control interface and DNS rebinding

Tavis Ormandy of Google Project Zero reported that the Transmission BitTorrent client's daemon, which accepted JSON-RPC requests on a local port, could be reached through a **DNS rebinding** attack: a hostname that first resolves to an attacker's server and later to `127.0.0.1`, so the browser believes it is still talking to the same origin. The fix added a check on the HTTP `Host` header, so that the daemon only answers requests addressed to names it expects.

Lesson: the same-origin policy is based on names, and DNS can change what a name points to. Services on local or private addresses must validate the `Host` header and require authentication. Chapter 18 shows how browsers have since added their own protections.

What the five cases have in common

CASE	WHAT WAS TRUSTED	THE DURABLE FIX
2016 research	Origins that merely <i>resembled</i> trusted ones	Exact, listed origins
2018 measurement	That CORS policies were correct by default	Testing policies as code, plus <code>Vary: Origin</code>
Samy, 2005	Everything running inside a trusted origin	Output encoding, content security policy
Zoom, 2019	Being reachable only from localhost	User confirmation and authentication
Transmission, 2018	That a name always means the same machine	<code>Host</code> validation and authentication

In every case, something was trusted because of *where a request seemed to come from*. Durable fixes replaced that assumption with an explicit decision.

CORS is not CSRF protection

CORS decides who may *read* a response. Cross-site request forgery is about requests that *change* something. Confusing the two leaves real holes.

The gap, stated plainly

Simple requests (Chapter 7) reach the server before any CORS check, often with the user's cookies. If one moves money or deletes a record, the server does it; the browser then hides the response, but the damage is done. That is **cross-site request forgery** (CSRF), and no CORS header prevents it.

WHAT CORS CONTROLS

- ✓ Whether a page may **read** a response
- ✓ Whether **non-simple** requests are sent (through the preflight)

Protects confidentiality of responses.

WHAT CORS DOES NOT CONTROL

- ✗ Whether a **simple** request is sent
- ✗ Whether the server **acts** on it
- ✗ Requests from outside a browser

Integrity of actions needs other defenses.

Figure 16.1 — Protecting what a request can *do* is a separate job.

Defenses that do protect state-changing endpoints

DEFENSE	HOW IT WORKS
SameSite cookies	<code>Lax</code> or <code>Strict</code> keeps the session cookie off cross-site requests. A strong default, but it is about <i>sites</i> , so sibling subdomains are not separated.
Anti-CSRF tokens	A secret per-session value in the server's own pages, required on every state change. Other sites cannot read it, so they cannot include it.
Require a non-simple request	Accept state changes only as strictly checked <code>application/json</code> or with a custom header. Forms cannot send those, and scripts must pass your preflight.
Check <code>Origin</code> on writes	Reject state-changing requests whose <code>Origin</code> is present and not on your list. Cheap, and effective against browser-based forgery.
Fetch Metadata	Modern browsers send <code>Sec-Fetch-Site</code> (<code>same-origin</code> , <code>same-site</code> , <code>cross-site</code> , <code>none</code>). A server can refuse cross-site requests to sensitive endpoints outright (Chapter 19).

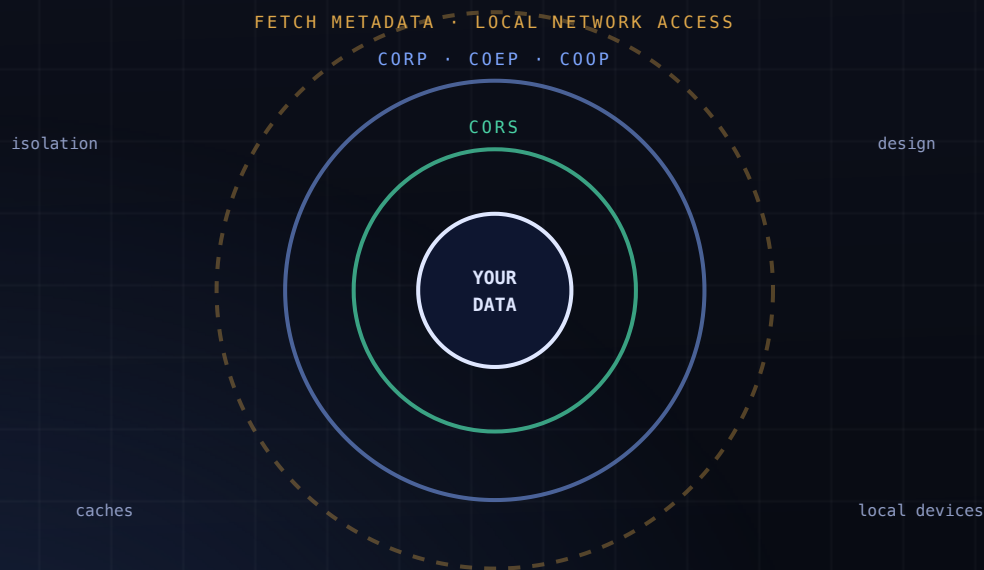
Layer them. Use SameSite cookies *and* at least one server-side check on every state-changing endpoint.

SECURITY REVIEW • 12:15 P.M.

A developer on the payments team admits the transfer endpoint also accepts form posts — “for an old mobile client.” Nobody says anything for a moment. Then the team adds a ninth ticket: **require JSON and check Origin on every write.**

Advanced Territory

Beyond the basics lie the places where CORS meets the rest of the platform: shared caches, the devices on your home and office networks, and a family of newer headers built after Spectre changed how browsers think about isolation. Then we put it all together into a design you can defend in any review.



Caches and Vary: Origin

The bug that makes CORS “work sometimes” is usually not in your CORS code at all. It is in a cache that doesn't know your answer depends on who asked.

How a correct policy goes wrong

Suppose Harbor Pay allows two origins, the dashboard and a partner portal. The API names whichever one is asking — a correct policy. Now put a CDN or reverse-proxy cache in front of it, keyed only on the URL:

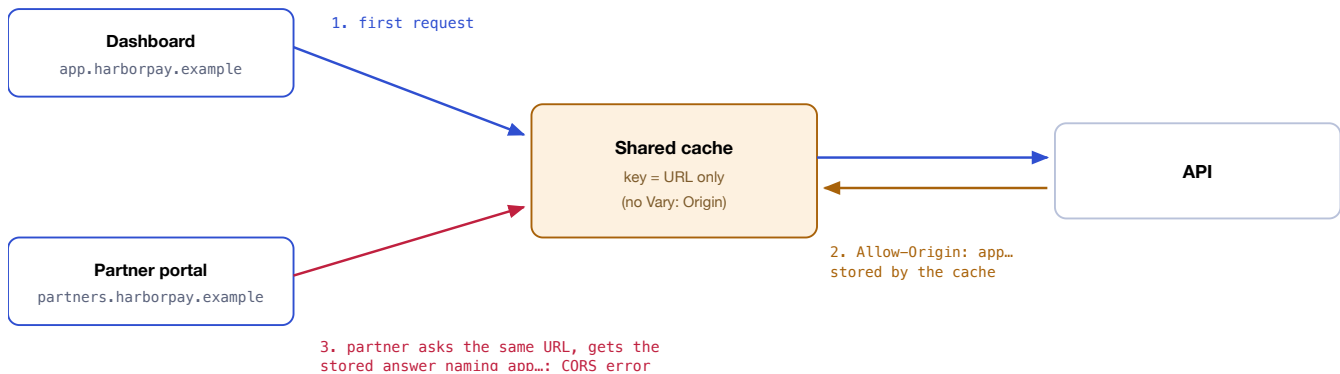


Figure 17.1 — A correct policy broken by a cache. Whichever origin asks first “wins,” so the bug comes and goes with cache expiry — the classic works-on-my-machine CORS report.

The fix: tell every cache what the answer depends on

Vary: Origin

Vary lists the request headers a response depends on. With **Vary: Origin**, a cache stores a separate copy per origin, and each origin gets its own correct answer. The rules are short:

- Send **Vary: Origin** whenever **Access-Control-Allow-Origin** can differ between requests — that is, on **every** response from a dynamic policy, including the ones where you send no CORS headers at all.
- A fixed **Access-Control-Allow-Origin: *** does not vary, so it doesn't need it.
- If your response already varies on other headers, combine them: **Vary: Origin, Accept-Encoding**.

Why it is a security issue too. The same mechanism can serve a response containing one user's data, or one origin's permission, to a different requester. Caches in front of personalized, credentialed responses need **Cache-Control: private** (or **no-store**) as well as correct **Vary**. The 2018 study in Chapter 15 found missing **Vary: Origin** to be widespread.

The local network

Your browser sits on two networks at once: the public internet, and the private one around you — your router, your printer, services on `localhost`. For most of the web's history, public pages could send requests to both.

Why the local network is special

Devices and apps on private networks were often built on the assumption that “only people on my network can reach me.” But every web page you open runs inside your browser, which *is* on your network. The Zoom and Transmission cases in Chapter 15 both followed from this. CORS helps only partly: it stops a public page from *reading* a local service's responses, but simple requests still arrive, and a local service that acts on them can be driven from any website.

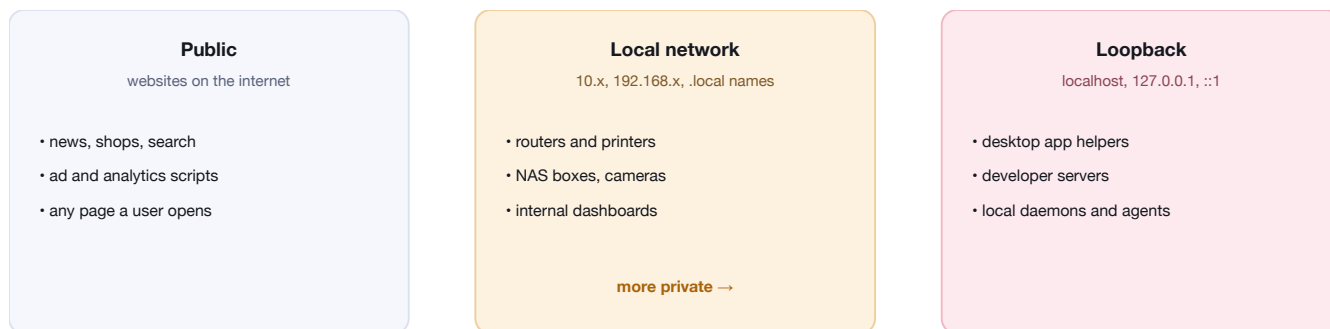


Figure 18.1 — Address spaces. Requests that move from a less private space to a more private one are the ones browsers now treat with extra suspicion.

What browsers did about it

Chromium first proposed **Private Network Access**: before a public page could reach a private address, the browser would send a special preflight (`Access-Control-Request-Private-Network: true`) and require the device to opt in. Deployment revealed compatibility problems, and the design evolved into **Local Network Access**. Beginning with Chrome 142 in October 2025, a public site that tries to reach the local network or loopback triggers a **permission prompt**: the user, not the device, decides whether that site may talk to local devices. Other browsers have been evaluating similar protections; check current documentation for each.

What this means for you. If your product legitimately talks to a local device from a web page (a printer setup page, a hardware wallet bridge, a developer tool), expect a permission prompt in Chromium browsers, explain it to your users, and never rely on the prompt as your only defense.

DNS rebinding, and why names are not enough

The same-origin policy compares *names*. DNS decides what machine a name refers to, and a domain's owner can change that answer at any time, even to a private address. That lets a hostname that began as an ordinary public site later point at `127.0.0.1`, while the browser still treats both as one origin. This is **DNS rebinding**, the issue behind the Transmission fix in Chapter 15. Browser-level defenses such as Local Network Access help, but the reliable fixes live in the local service itself.

Building a local service that is safe to run

PRACTICE	WHY IT MATTERS
Validate the <code>Host</code> header	Answer only requests addressed to names you expect (<code>localhost</code> , <code>127.0.0.1</code> , your device name). A rebound hostname arrives with the attacker's domain in <code>Host</code> , so this check defeats DNS rebinding.
Require authentication	A random secret generated at install time, known to the legitimate client, turns “anyone who can reach me” into “anyone who holds the key.”
Never act on simple requests	Require JSON bodies or a custom header for anything with side effects, so browsers must preflight — and then refuse preflights from origins you don't expect.
Confirm with the user	Actions with real consequences (joining a call, sending a file, changing settings) should need a click inside your own app, not just a request.
Bind to loopback, and remove on uninstall	Listen on <code>127.0.0.1</code> , not all interfaces, and make sure uninstalling the product really removes the service.
Check <code>Origin</code> and Fetch Metadata	Refuse browser requests whose <code>Origin</code> or <code>Sec-Fetch-Site</code> shows they came from an unexpected website.

SECURITY REVIEW · 2:10 P.M.

Harbor Pay ships a desktop helper for card readers at its retail partners. It listens on `localhost` . It has no authentication, because “*it's only local.*” The room goes quiet for the second time that day.

Ticket ten: **Host validation, install-time secret, user confirmation for pairing.**

Principle. “Only reachable locally” is a statement about the network. Every web page the user opens is, in effect, local too. Design local services as if they were on the internet, because through the browser, they are.

Beyond CORS

CORS controls what a page may read through `fetch()`. A newer family of headers controls everything else: what may embed you, what you may embed, and who may share your process.

The turning point: Spectre

In January 2018, researchers disclosed **Spectre**, a class of CPU vulnerabilities that let code infer data from memory it was never allowed to read. For browsers the implication was stark: if a cross-origin resource was loaded into the same process as a hostile page — even as an “embedded” image or script it could not officially read — its bytes might be exposed. Browsers responded with **Site Isolation** (separate processes per site) and by blocking sensitive responses from ever reaching the wrong process: first **Cross-Origin Read Blocking** (CORB), now specified as **Opaque Response Blocking** (ORB) in the Fetch Standard. Servers got new headers to state their intentions explicitly.

HEADER	QUESTION IT ANSWERS	TYPICAL VALUE
Cross-Origin-Resource-Policy (CORS)	Who may embed this resource with a no-cors load, such as an image or script tag?	<code>same-origin</code> for private resources; <code>same-site</code> ; or <code>cross-origin</code> for public assets.
Cross-Origin-Embedder-Policy (COEP)	May this page load cross-origin resources that haven't opted in?	<code>require-corp</code> (every subresource must allow it via CORS or CORP), or <code>credentialless</code> .
Cross-Origin-Opener-Policy (COOP)	May cross-origin windows I open, or that open me, keep a reference to me?	<code>same-origin</code> isolates the page into its own browsing context group.

Cross-origin isolation

A page served with both `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp` (or `credentialless`) becomes **cross-origin isolated**: `self.crossOriginIsolated` is `true`. Browsers then unlock powerful features they otherwise restrict, such as `SharedArrayBuffer` and more precise timers, because nothing from another origin can end up in the page's process without having agreed to it.

How CORS fits. Under COEP: `require-corp`, a cross-origin resource is accepted if it either sends a suitable CORP header or is fetched in CORS mode and passes the CORS check. CORS and CORP are the two ways a resource can say “yes, you may have me.”

Fetch Metadata: the browser tells you the context

Modern browsers attach **Fetch Metadata** request headers that describe how a request came about. A page cannot forge them.

HEADER	TELLS THE SERVER
Sec-Fetch-Site	The relationship between the requesting page and the target: <code>same-origin</code> , <code>same-site</code> , <code>cross-site</code> , or <code>none</code> (typed in the address bar, a bookmark).
Sec-Fetch-Mode	<code>navigate</code> , <code>cors</code> , <code>no-cors</code> , <code>same-origin</code> or <code>websocket</code> .
Sec-Fetch-Dest	What the response will become: <code>document</code> , <code>image</code> , <code>script</code> , <code>empty</code> (fetch), and so on.
Sec-Fetch-User	<code>?1</code> when a navigation was triggered by a user action.

A resource isolation policy

With these headers, a server can refuse unexpected cross-site traffic before it reaches any business logic — a strong complement to CORS (which governs reading) and to CSRF defenses (which govern writing):

```
// Allow same-origin and same-site requests, browser navigations, and old browsers
// that don't send the header. Refuse everything else from other sites.
function allowRequest(req) {
  const site = req.headers['sec-fetch-site'];
  if (!site) return true; // older browser
  if (['same-origin', 'same-site', 'none'].includes(site)) return true;
  if (req.headers['sec-fetch-mode'] === 'navigate' && req.method === 'GET') return true;
  return isPublicEndpoint(req.url); // explicit exceptions
}
```

Cross-site leaks: the frontier

Even when a page cannot read a response, it can sometimes observe side effects: how long a load took, whether a frame loaded, how many windows a site opened. The research community calls these **cross-site leaks** (XS-Leaks). They are the reason browsers keep adding isolation features and keep CORS errors deliberately vague (Chapter 10). The defenses are the ones in this chapter: `SameSite` cookies, Fetch Metadata checks, COOP, and CORP.

For APIs

Precise CORS allowlist,
Vary: `Origin`, Fetch Metadata
policy, CORP: `same-site` on
private JSON.

For apps

COOP: `same-origin`, a strict
content security policy, and
`SameSite` session cookies.

For public assets

`Allow-Origin: *` without
credentials, and
CORP: `cross-origin` so
isolated pages can use them.

Designing a secure CORS policy

The safest CORS policy is the one you never need. The next safest is short, explicit, and boring. Here is how to choose, and how to write it.

Three architectures

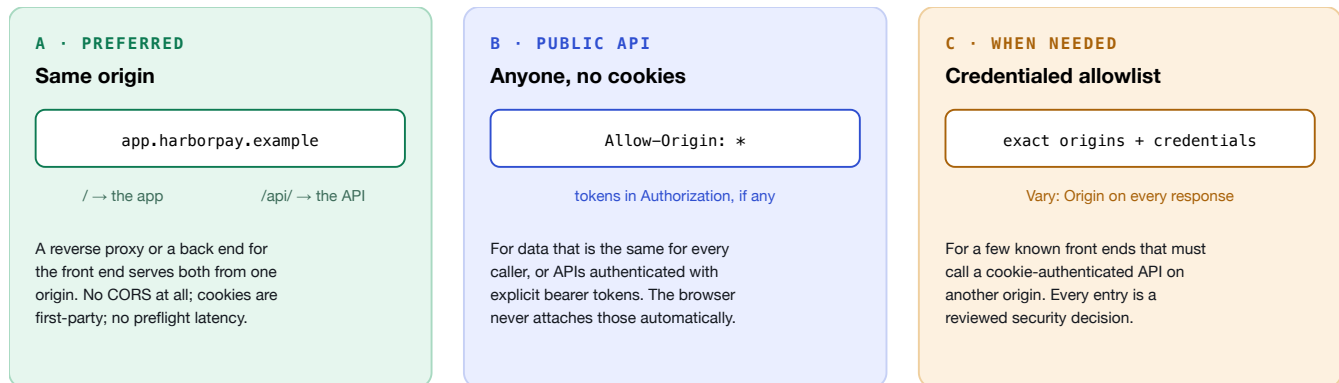


Figure 20.1 — Choose the architecture first; the headers follow from it. Harbor Pay's dashboard used C on launch night and moved to A the following quarter.

Rules for a credentialed allowlist

1. **Exact strings only.** Full origins (`https://host` , plus port if not default), compared with equality. No substring, prefix, suffix or regular-expression checks.
2. **Never `null` , never `http`** in production, and no development origins in production configuration.
3. **Headers only on a match.** Unknown origins get no `Access-Control-*` headers at all — not an error message, not a default origin.
4. **`Vary: Origin` on every response** the policy touches, allowed or not.
5. **Minimal methods and headers**, listed by name; `Max-Age` of minutes to an hour.
6. **Preflights answered before authentication**, and never running business logic.
7. **Expose only what the client needs** in `Access-Control-Expose-Headers` .
8. **Configured in one place** — app or gateway, not both.

Reference configurations

Three stacks, one allowlist. Check your framework version's documentation first.

Node.js with Express and the cors package

```
const cors = require('cors');
app.use('/api', cors({
  origin: ['https://app.harborpay.example'], // an exact list, never true or '*'
  credentials: true,
  methods: ['GET', 'POST', 'PUT'],
  allowedHeaders: ['Content-Type'],
  exposedHeaders: ['X-Request-Id'],
  maxAge: 600,
}));
```

Spring (Java) with WebMvcConfigurer

```
@Override
public void addCorsMappings(CorsRegistry registry) {
  registry.addMapping("/api/**")
    .allowedOrigins("https://app.harborpay.example")
    .allowedMethods("GET", "POST", "PUT")
    .allowedHeaders("Content-Type")
    .exposedHeaders("X-Request-Id")
    .allowCredentials(true)
    .maxAge(600);
}
```

nginx in front of any application

```
# Unknown origins map to "", and nginx omits empty headers.
map $http_origin $cors_origin {
  default "";
  "https://app.harborpay.example" $http_origin;
}
location /api/ {
  add_header Access-Control-Allow-Origin $cors_origin always;
  add_header Access-Control-Allow-Credentials "true" always;
  add_header Vary Origin always;
  if ($request_method = OPTIONS) {
    add_header Access-Control-Allow-Origin $cors_origin always;
    add_header Access-Control-Allow-Credentials "true" always;
    add_header Access-Control-Allow-Methods "GET, POST, PUT" always;
    add_header Access-Control-Allow-Headers "Content-Type" always;
    add_header Access-Control-Max-Age 600 always;
    add_header Vary Origin always;
    return 204;
  }
  proxy_pass http://api_backend;
}
```

Inner `add_header` blocks replace inherited ones, hence the repetition.

Test the policy like code

Policies drift: a new front end is added in a hurry, a gateway is upgraded, someone “temporarily” adds a test origin. Put the policy under automated tests so drift fails a build instead of reaching production. A useful suite sends preflight and actual requests with a range of `Origin` values and asserts on the headers that come back:

REQUEST WITH ORIGIN...	THE POLICY MUST RESPOND WITH...
each origin on the allowlist	that exact origin, <code>Allow-Credentials: true</code> if used, <code>Vary: Origin</code>
an origin <i>not</i> on the list	no <code>Access-Control-Allow-Origin</code> at all, and still <code>Vary: Origin</code>
the <code>http://</code> form of an allowed origin	no <code>Access-Control-Allow-Origin</code>
<code>null</code>	no <code>Access-Control-Allow-Origin</code>
an allowed origin, on a 404 or 500 response	the same CORS headers as on success
an allowed origin, preflight with an unlisted header	no permission for that header

The lab server's `node server.js check` is the smallest possible version of this idea; Lab 8 asks you to extend it.

The design-review checklist

- ☐ Could this be same-origin instead (architecture A)?
- ☐ Does any response depend on cookies or network location?
- ☐ If `*` is used, are responses identical for every caller?
- ☐ Is every allowed origin a full, exact `https` string?
- ☐ No `null`, no `http`, no dev origins in production?
- ☐ Does every trusted origin have XSS defenses and a named owner?
- ☐ `Vary: Origin` on every response of a dynamic policy?
- ☐ Preflights answered before authentication, no side effects?
- ☐ Methods, headers and exposed headers listed minimally, by name?
- ☐ CORS headers present on error responses too?
- ☐ Configured in exactly one layer?
- ☐ State-changing endpoints protected against CSRF (Chapter 16)?
- ☐ Local services validate `Host` and authenticate (Chapter 18)?
- ☐ Automated tests cover allowed and refused origins?

If you remember one design rule. Let the browser say no by default, and make every yes a named, reviewed, tested decision. That is what `Access-Control-Allow-Origin` was designed to express.

Epilogue: the review

Every story about security ends the same way: not with a dramatic rescue, but with a list of boring things done well.

HARBOR PAY • SIX WEEKS AFTER LAUNCH

The ten tickets are closed. The five services that checked whether an origin *ended with* the company domain now compare against exact lists. The two that trusted `null` have lost their decade-old test harness. The reporting service that echoed every origin turned out to need exactly one: the finance dashboard. The transfer endpoint accepts only JSON and checks `Origin` on every write. The card-reader helper asks the cashier to confirm pairing and ignores requests addressed to any unexpected host name.

The bigger change is quieter. The dashboard team has moved the API behind the app's own origin, at `app.harborpay.example/api`. For the dashboard, CORS is simply gone — no preflights, no allowlist to maintain, first-party cookies. The build pipeline now runs a CORS test suite against every service, and a failing test names the origin and the header.

On a Friday afternoon, a new engineer posts in the team channel: *“Getting a CORS error on the partner portal — can I just set Allow-Origin to star?”*

You smile, and start typing. *“Before anybody touches that header,”* you write, *“can you tell me what it actually protects?”*

Seven things to carry with you

1. **The same-origin policy is the default;** CORS is a set of exceptions to it, granted by servers and enforced by browsers.
2. **It governs reading, not sending.** Simple requests reach the server regardless; protect state changes separately.
3. **An origin is scheme, host and port.** Sibling subdomains and different ports are different origins.
4. **Credentials raise the bar:** an exact origin, `Allow-Credentials: true`, and no wildcards — and that bar exists for good reason.
5. **Compare origins with exact equality** against a reviewed list. Every clever rule is a future incident.
6. **Send `Vary: Origin`** whenever the answer depends on the asker.
7. **The best CORS policy is often none at all:** serve the API from the app's origin.

Now run it. Part V turns each of these ideas into an experiment you can watch in your own browser's DevTools. Eight labs, one small server, about an hour.

The Labs

One small server, two origins on your own machine, and your browser's DevTools. Each lab changes one line of the CORS policy and asks you to predict what the browser will do before you look. By the end, the error messages from Chapter 11 will read like plain English.

Network	Console	Application	
NAME	METHOD	STATUS	TYPE
rates	GET	200	fetch
notes	OPTIONS	204	preflight
notes	POST	201	fetch
profile	GET	CORS error	fetch
access-control-allow-origin: http://localhost:8080			
access-control-allow-methods: GET, POST, PUT, DELETE			
vary: Origin			

Setup · 1–3 First contact · 4–6 Credentials and headers · 7–8 Caching and allowlists

Listing A lab/server.js, complete

The lab server

Everything runs on your own machine. Nothing is installed, nothing is downloaded, and nothing listens beyond `localhost`.

What you need

- **Node.js 18 or later.** Check with `node --version`.
- **A modern browser** with DevTools, open to the **Network** and **Console** tabs.
- **The file `server.js`**, from this book's companion folder or typed in from Listing A at the end of this part. It is a single file with no dependencies.

Start it

```
$ node server.js
Harbor Pay app http://localhost:8080
Harbor Pay API http://localhost:8081 (request log: /log)
Allowed origins: http://localhost:8080
```

What is running

ADDRESS	WHAT IT IS
<code>http://localhost:8080</code>	The app origin : one page with four buttons, each making a different cross-origin <code>fetch()</code> to the API.
<code>http://localhost:8081</code>	The API origin : <code>/api/rates</code> (public), <code>/api/profile</code> (needs a session cookie), <code>/api/notes</code> (JSON <code>POST</code>), and <code>/login</code> , which sets the cookie.
<code>http://localhost:8081/log</code>	The last 40 requests the API <i>received</i> : your proof of what really reached the server.

The two addresses differ only in port: **different origins** but the **same site** (Chapter 3), so no third-party cookie settings get in the way.

How every lab works

1. Open the file and find the `POLICY` object near the top.
2. Make the lab's one-line change and restart (`Ctrl+C`, `node server.js`).
3. **Predict** the result, then press the button and check the page, Console, Network tab and `/log`. Undo the change before the next lab.

`node server.js check` shows the policy without a browser: it sends preflights from an allowed and an unlisted origin and prints the headers each receives.

For learning only: plain HTTP, in-memory data and a deliberately simple sign-in. Never expose it to a network or reuse it as production code.

Labs 1–3: first contact

Watch the basic exchange from Chapter 6, the preflight from Chapter 8, and the single most important fact about CORS: a blocked request still arrives.

LAB 1 • CHAPTER 6

The permission slip

open `http://localhost:8080` → press “Lab 1”

In the Network tab, select the `rates` request. Under **Request Headers**, find `Origin: http://localhost:8080`, added by the browser. Under **Response Headers**, find `Access-Control-Allow-Origin` naming the same origin. The page shows:

```
Lab 1: status 200
x-request-id visible to script: req-6abac081
{ "USD": 1, "EUR": 0.92, "GBP": 0.79 }
```

Notice: there is no `OPTIONS` row. A plain `GET` is a simple request (Chapter 7).

LAB 2 • CHAPTER 8

The preflight

press “Lab 2” → then open `http://localhost:8081/log`

This button sends a `JSON POST`. The Network tab now shows **two** rows for `notes`: an `OPTIONS` preflight (status 204) and then the `POST` (status 201). Compare the preflight's

`Access-Control-Request-Headers: content-type` with the answer's `Access-Control-Allow-Headers`.

The log (newest first) confirms the order:

```
POST /api/notes origin http://localhost:8080
OPTIONS /api/notes origin http://localhost:8080 preflightFor POST
```

Try this: in the Network tab, check that the preflight carried no `Cookie` header, even after Lab 4 has signed you in.

LAB 3 • CHAPTERS 4 AND 10

Blocked, but delivered

open `http://127.0.0.1:8080` → press “Lab 1”

`127.0.0.1` and `localhost` reach the same server, but they are different hosts, so this page has a **different origin** that is not on the allowlist. The page reports only a generic failure:

```
Lab 1: TypeError: Failed to fetch
The script cannot tell why. Check the Console and Network tabs.
```

The Console explains that no `Access-Control-Allow-Origin` header was present. Now open `/log`: the `GET /api/rates` from `http://127.0.0.1:8080` **is there**. The server received it and answered; the browser withheld the answer.

Why it matters: this is Chapter 16 in one screen. Had this request changed data, the change would have happened.

Labs 4–6: credentials and headers

The strict rules of Chapter 9, and the two header lists that trip up almost every team at least once.

LAB 4 • CHAPTER 9

Two opt-ins for credentials

press “Lab 3” → visit `http://localhost:8081/login` → press “Lab 3” again

The first press returns `401 not signed in` — readable, because the policy allows this origin. After visiting `/login`, the browser holds a session cookie for the API, and the page's `credentials: 'include'` sends it:

```
Lab 3: status 200
{ "user": "demo-user", "plan": "Harbor Pay Plus" }
```

Now change `allowCredentials: true` to `false` and restart. The same button fails, and the Console says `Access-Control-Allow-Credentials must be 'true'`. The cookie was sent and the server answered; the browser refused to share a credentialed response the server hadn't agreed to share.

LAB 5 • CHAPTER 10

Visible in DevTools, invisible to code

press “Lab 4”, then set `exposeHeaders: []` and press it again

With the default policy the page prints `x-request-id visible to script: req-...`. After emptying `exposeHeaders` and restarting, it prints:

```
x-request-id visible to script: (null)
```

Yet the Network tab still shows `x-request-id` in the response headers. The browser received it; the page was not allowed to read it.

LAB 6 • CHAPTERS 8 AND 11

A header the preflight didn't approve

remove `'content-type'` from `allowHeaders` → press “Lab 2”

The preflight still returns 204, but its `Access-Control-Allow-Headers` no longer lists `content-type`. The browser stops there: the `POST` is **never sent**, and `/log` shows only the `OPTIONS` row. The Console names the header that wasn't allowed.

Compare with Lab 3: there, a simple request arrived anyway. Here, the preflight protected the server from a request it hadn't agreed to receive. That difference is the whole reason preflights exist.

Labs 7–8: caching and allowlists

Measure what a preflight costs, then change the allowlist the right way and put it under test.

LAB 7 • CHAPTER 8

Counting preflights

```
press "Lab 2" five times → count OPTIONS rows in /log
```

With `maxAgeSeconds: 600`, you should see a single `OPTIONS` followed by five `POST`s: the browser cached the preflight answer. Now set `maxAgeSeconds: 0`, restart, open the page in a fresh private window, and repeat. Every `POST` is now preceded by its own `OPTIONS` — twice the round trips.

Tip: the preflight cache belongs to the browser session. If results look stale, use a fresh private window; and leave DevTools' *Disable cache* setting off for this lab so you are measuring the browser's normal behavior.

LAB 8 • CHAPTERS 13, 17 AND 20

Adding an origin, properly

```
node server.js check
```

First read the output: the preflight from `http://localhost:8080` gets the full set of permission headers; the one from `http://127.0.0.1:8080` gets only `Vary: Origin`. Now add `'http://127.0.0.1:8080'` to `allowedOrigins`, restart, and repeat Lab 3: it succeeds.

Preflight from an origin not on the list

```
OPTIONS /api/notes Origin: http://127.0.0.1:8080
→ 204
```

```
vary: Origin
```

```
(no Access-Control-Allow-Origin: the browser will not share the response)
```

Extend it: turn the `check` function into the test suite from Chapter 20. Add probes for `Origin: null` and for an `https` version of the app origin, and make the script exit with an error if any of them receives an `Access-Control-Allow-Origin` header. You have just written the CI check that ended Harbor Pay's story.

Where to go next

Put the lab server behind a reverse proxy that serves both the page and the API from `http://localhost:8080` (architecture A from Chapter 20). Watch every CORS header and every preflight disappear from the Network tab — the most satisfying lab of all.

The lab server, complete

Both origins, the request log and the CORS policy in one file. Every lab in Part V is a one-line edit to the `POLICY` object near the top; `applyCors` is the whole of Chapter 20's advice as running code.

LINES 1–53 OF 225

```

1  'use strict';
2  // =====
3  // cors-guide lab - server.js          run:  node server.js
4  //
5  // Two origins on your own machine, and nothing else:
6  //
7  //   http://localhost:8080   the Harbor Pay web app (a page with lab buttons)
8  //   http://localhost:8081   the Harbor Pay API, which applies the CORS policy
9  //
10 // The CORS policy is the POLICY object below. Every lab in the book asks you
11 // to change one line of it, restart, and watch the browser react in DevTools.
12 //
13 //   node server.js          start both origins
14 //   node server.js check    print the headers the policy produces, and exit
15 //
16 // Zero dependencies: Node 18 or later, core modules only.
17 // =====
18 const http = require('node:http');
19
20 const APP_PORT = 8080;
21 const API_PORT = 8081;
22
23 // --- the policy -----
24 // An exact allowlist. No patterns, no echoing, no "*" alongside credentials.
25 const POLICY = {
26   allowedOrigins: ['http://localhost:8080'],
27   allowMethods: ['GET', 'POST', 'PUT', 'DELETE'],
28   allowHeaders: ['content-type', 'x-request-id'],
29   exposeHeaders: ['x-request-id'],
30   allowCredentials: true,
31   maxAgeSeconds: 600,
32 };
33
34 // --- the CORS middleware -----
35 // Returns true when it has fully answered the request (a preflight).
36 function applyCors(req, res) {
37   const origin = req.headers.origin;
38
39   // The answer depends on the Origin header, so any cache in between must
40   // key on it too. Sent on every response, allowed or not.
41   res.setHeader('Vary', 'Origin');
42
43   const allowed = origin !== undefined && POLICY.allowedOrigins.includes(origin);
44
45   if (allowed) {
46     res.setHeader('Access-Control-Allow-Origin', origin); // one exact origin
47     if (POLICY.allowCredentials) res.setHeader('Access-Control-Allow-Credentials', 'true');
48     if (POLICY.exposeHeaders.length) {
49       res.setHeader('Access-Control-Expose-Headers', POLICY.exposeHeaders.join(', '));
50     }
51   }
52
53   const isPreflight = req.method === 'OPTIONS'

```

The lab server, complete

LINES 54–111 OF 225

```

54     && req.headers['access-control-request-method'] !== undefined;
55     if (!isPreflight) return false;
56
57     if (allowed) {
58         res.setHeader('Access-Control-Allow-Methods', POLICY.allowMethods.join(', '));
59         res.setHeader('Access-Control-Allow-Headers', POLICY.allowHeaders.join(', '));
60         res.setHeader('Access-Control-Max-Age', String(POLICY.maxAgeSeconds));
61     }
62     // A refused preflight is simply one without the permission headers.
63     res.writeHead(204);
64     res.end();
65     return true;
66 }
67
68 // --- a request log, so you can see what reached the server -----
69 const LOG = [];
70 function record(req) {
71     LOG.unshift({
72         at: new Date().toISOString().slice(11, 19),
73         method: req.method,
74         path: req.url,
75         origin: req.headers.origin || '(none)',
76         preflightFor: req.headers['access-control-request-method'] || '',
77         cookie: req.headers.cookie ? 'yes' : 'no',
78     });
79     LOG.length = Math.min(LOG.length, 40);
80 }
81
82 // --- the API origin -----
83 const NOTES = [];
84 let nextId = 1;
85
86 function json(res, status, body, extra = {}) {
87     res.writeHead(status, { 'content-type': 'application/json', ...extra });
88     res.end(JSON.stringify(body, null, 2));
89 }
90
91 function readBody(req) {
92     return new Promise((resolve) => {
93         let data = '';
94         req.on('data', (c) => { data += c; });
95         req.on('end', () => resolve(data));
96     });
97 }
98
99 const sessionOf = (req) => /(?:^(|;s*)hp_session=(|;|+)|)/.exec(req.headers.cookie || '');
100
101 const api = http.createServer(async (req, res) => {
102     record(req);
103     if (applyCors(req, res)) return;
104     const url = new URL(req.url, 'http://localhost');
105     const requestId = 'req-' + Math.random().toString(16).slice(2, 10);
106
107     if (url.pathname === '/api/rates' && req.method === 'GET') {
108         return json(res, 200, { USD: 1, EUR: 0.92, GBP: 0.79 }, { 'x-request-id': requestId });
109     }
110
111     if (url.pathname === '/login' && req.method === 'GET') {

```

The lab server, complete

LINES 112–168 OF 225

```

112 // Visit this URL directly in the browser to get a session cookie.
113 res.setHeader('Set-Cookie', 'hp_session=demo-user; Path=/; Max-Age=3600; HttpOnly; SameSite=Lax');
114 return json(res, 200, { signedIn: 'demo-user', next: 'return to http://localhost:8080' });
115 }
116
117 if (url.pathname === '/api/profile' && req.method === 'GET') {
118   const session = sessionOf(req);
119   if (!session) return json(res, 401, { error: 'not signed in' });
120   return json(res, 200, { user: session[1], plan: 'Harbor Pay Plus' },
121     { 'x-request-id': requestId });
122 }
123
124 if (url.pathname === '/api/notes' && req.method === 'POST') {
125   if (!(req.headers['content-type'] || '').startsWith('application/json')) {
126     return json(res, 415, { error: 'send application/json' });
127   }
128   const note = { id: nextId++, text: String(JSON.parse(await readBody(req)).text || '') };
129   NOTES.push(note);
130   return json(res, 201, note, { 'x-request-id': requestId });
131 }
132
133 if (url.pathname === '/log') {
134   return json(res, 200, LOG);
135 }
136
137 json(res, 404, { error: 'not found' });
138 });
139
140 // --- the app origin -----
141 const PAGE = `<!doctype html>
142 <html lang="en-US"><head><meta charset="utf-8"><title>Harbor Pay - CORS lab</title>
143 <style>
144   body { font: 15px/1.5 system-ui, sans-serif; max-width: 760px; margin: 32px auto; padding: 0 16px; }
145   button { font: inherit; margin: 4px 6px 4px 0; padding: 6px 12px; cursor: pointer; }
146   pre { background: #f4f6fb; padding: 12px; border-radius: 6px; white-space: pre-wrap; }
147   .ok { color: #0d7a52; } .no { color: #c0203f; }
148 </style></head><body>
149 <h1>Harbor Pay &mdot; CORS lab</h1>
150 <p>This page is served from <b id="me"></b>. The API lives at <b>http://localhost:8081</b>.
151 Open DevTools (Network and Console tabs) before pressing a button.</p>
152 <button data-lab="1">Lab 1 &mdot; GET public rates</button>
153 <button data-lab="2">Lab 2 &mdot; POST JSON (preflight)</button>
154 <button data-lab="3">Lab 3 &mdot; GET profile with cookie</button>
155 <button data-lab="4">Lab 4 &mdot; read an exposed header</button>
156 <pre id="out">Results appear here.</pre>
157 <script>
158 const API = 'http://localhost:8081';
159 document.getElementById('me').textContent = location.origin;
160 const out = document.getElementById('out');
161 const show = (cls, text) => { out.className = cls; out.textContent = text; };
162
163 const LABS = {
164   1: () => fetch(API + '/api/rates'),
165   2: () => fetch(API + '/api/notes', { method: 'POST',
166     headers: { 'content-type': 'application/json' },
167     body: JSON.stringify({ text: 'hello from ' + location.origin }) }),
168   3: () => fetch(API + '/api/profile', { credentials: 'include' }),

```

The lab server, complete

LINES 169–225 OF 225

```

169   4: () => fetch(API + '/api/rates'),
170   };
171
172   async function run(n) {
173     try {
174       const res = await LABS[n]();
175       const body = await res.text();
176       const rid = res.headers.get('x-request-id');
177       show('ok', 'Lab ' + n + ': status ' + res.status + '\n'
178         + 'x-request-id visible to script: ' + (rid || '(null)') + '\n\n' + body);
179     } catch (err) {
180       // The browser deliberately says almost nothing here. The Console has the detail.
181       show('no', 'Lab ' + n + ': ' + err.name + ': ' + err.message
182         + '\n\nThe script cannot tell why. Check the Console and Network tabs.');
```

Glossary

Every term in one place, with the chapter that explains it.

Ambient authority

Credentials such as cookies that the browser attaches automatically, whoever caused the request. (Ch. 2)

Access-Control-Allow-Origin

The response header naming the one origin, or `*`, allowed to read the response. (Ch. 6)

CORP, COEP, COOP

Cross-Origin Resource, Embedder and Opener Policies: headers controlling embedding and process isolation. (Ch. 19)

CORS

Cross-Origin Resource Sharing: the protocol by which a server lets the browser share its responses with named other origins. (Ch. 6)

CORS-safelisted

The request headers, content types and response headers that need no special permission. (Ch. 7, 10)

Credentials

Cookies, HTTP authentication and TLS client certificates. Sent cross-origin only with `credentials: 'include'`. (Ch. 9)

Cross-origin isolation

The state of a page served with COOP and COEP, which unlocks features such as `SharedArrayBuffer`. (Ch. 19)

CSRF

Cross-site request forgery: causing a user's browser to send a state-changing request they didn't intend. Not prevented by CORS. (Ch. 16)

DNS rebinding

Changing what a hostname resolves to, so a browser treats a local address as the same origin as a public site. (Ch. 15, 18)

Fetch Metadata

The `Sec-Fetch-*` request headers describing how a request came about. (Ch. 19)

Fetch Standard

The WHATWG living standard that defines fetching, including CORS. The authoritative source. (Ch. 6)

JSONP

A pre-CORS technique that delivered data as an executable script. Obsolete. (Ch. 5)

Local Network Access

Chromium's permission-based protection for requests from public sites to local and loopback addresses. (Ch. 18)

Opaque origin

An origin with no identity, serialized as `null`; never equal to any other origin. (Ch. 3)

Opaque response

The result of a `no-cors` fetch: usable by the browser, unreadable by script. (Ch. 10)

Origin

The triple of scheme, host and port that the same-origin policy compares. (Ch. 3)

ORB / CORB

Opaque (formerly Cross-Origin) Read Blocking: stops sensitive responses from entering another site's process. (Ch. 19)

Preflight

The `OPTIONS` request a browser sends to ask permission before a non-simple request. (Ch. 8)

Public Suffix List

The list of suffixes, like `.com` or `.github.io`, under which domains are registered; it defines sites. (Ch. 3)

Same-origin policy

The browser rule that a page may read only responses from its own origin, unless CORS permits otherwise. (Ch. 2, 4)

SameSite

A cookie attribute that limits sending the cookie on cross-site requests. (Ch. 9, 16)

Simple request

A request a form could send, which needs no preflight. (Ch. 7)

Site

Scheme plus registrable domain. Coarser than an origin; used by cookies. (Ch. 3)

Vary

The response header listing the request headers a response depends on; `Vary: Origin` keeps caches correct. (Ch. 17)

XS-Leaks

Cross-site leaks: information inferred from side effects of cross-origin loads. (Ch. 19)

Further reading

The specifications, the best guides, and the sources behind the case studies.

Read these first

SOURCE	WHY READ IT
WHATWG Fetch Standard fetch.spec.whatwg.org	The normative definition of CORS, preflights, credentials, opaque responses and ORB.
RFC 6454 , <i>The Web Origin Concept</i> rfc-editor.org/rfc/rfc6454	The definition of an origin and the <code>Origin</code> header.
MDN Web Docs , CORS guide developer.mozilla.org	The most readable practical reference, with browser compatibility tables.
OWASP WSTG-CLNT-07 owasp.org/www-project-web-security-testing-guide	How security testers assess a CORS policy — useful for writing your own tests.

Going deeper

CWE-942, CWE-346 cwe.mitre.org	The weakness entries for permissive cross-domain policies and origin validation errors.
Fetch Metadata and cross-origin isolation web.dev	Google's guides to resource isolation policies, COOP and COEP.
Local Network Access developer.chrome.com/blog/local-network-access	Chromium's announcement of the permission prompt that replaced Private Network Access preflights.
XS-Leaks Wiki xsleaks.dev	A catalog of cross-site leak techniques and their defenses.

Sources for the case studies

- J. Kettle, “Exploiting CORS Misconfigurations for Bitcoins and Bounties,” OWASP AppSec USA, October 2016; PortSwigger Research.
- J. Chen et al., “We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS,” 27th USENIX Security Symposium, August 2018.
- J. Leitschuh, disclosure of the Zoom macOS local web server issue, July 2019; CVE-2019-13450.
- T. Ormandy, report on the Transmission RPC daemon and DNS rebinding, January 2018; CVE-2018-5702.
- J. Grossman, “Advanced Web Attack Techniques using Gmail,” January 2006.
- B. Chess, Y. Tsipenyuk O'Neil, J. West, “JavaScript Hijacking,” Fortify Software, 2007.
- Public reporting on the MySpace “Samy” worm, October 2005.

Browser security moves quickly. When a detail matters to a design decision, confirm it against the Fetch Standard and current browser documentation.

“Before anybody touches that header, can someone tell me what it actually protects?”

Every web developer meets CORS as a red error in the console. Most learn just enough to make it go away — and some of those quick fixes quietly expose every signed-in user of an API to every website on the internet.

This book follows a fintech team from a failed launch night to a company-wide security review, and teaches CORS the way it should be taught: from the same-origin policy up. You will learn:

- why browsers built the wall, and exactly what it blocks
- simple requests, preflights, credentials and exposed headers
- a step-by-step method for debugging any CORS error
- the six configuration mistakes behind real incidents
- seven attack patterns, drawn, with warning signs and defenses
- caches, local networks, and the isolation headers after Spectre
- architectures, reference configurations and a review checklist

Eight hands-on labs use a single zero-dependency Node.js server, printed in full, so you can watch every rule in your own browser's DevTools.

AUTHOR

Smiroh Dev

Computers / Security / Web Development

SMIROH PUBLISHERS · FIRST EDITION